
ITHub
Поведенческие шаблоны
Java

как объекты обмениваются информацией

передают ответственность

выполняют действия

реагируют на события

**Поведенческие шаблоны проектирования** — это типовые решения, которые помогают организовать взаимодействие между объектами программы.

# Основные поведенческие паттерны

Chain of Responsibility (цепочка обязанностей)

Mediator (посредник)

Memento (хранитель или снимок)

Command (команда)

Observer (наблюдатель)

Interpreter (интерпретатор)

State (состояние)

Iterator (итератор)

Strategy (стратегия)

Template Method (шаблонный метод)

Visitor (посетитель)

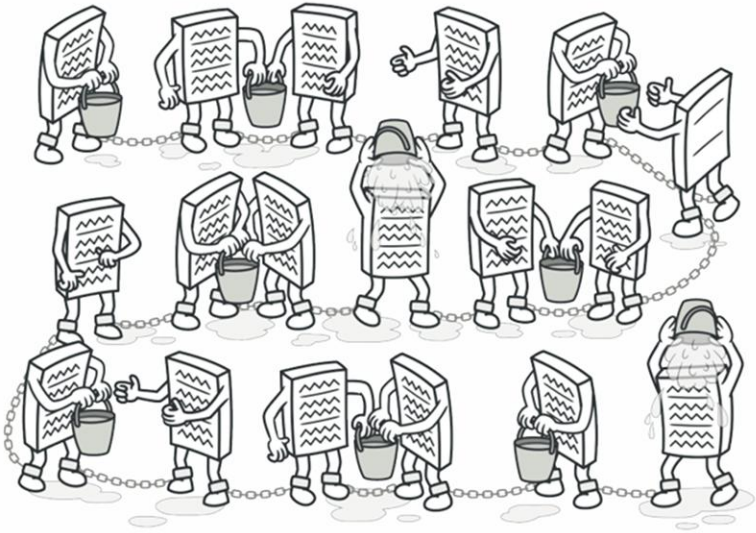
IThub

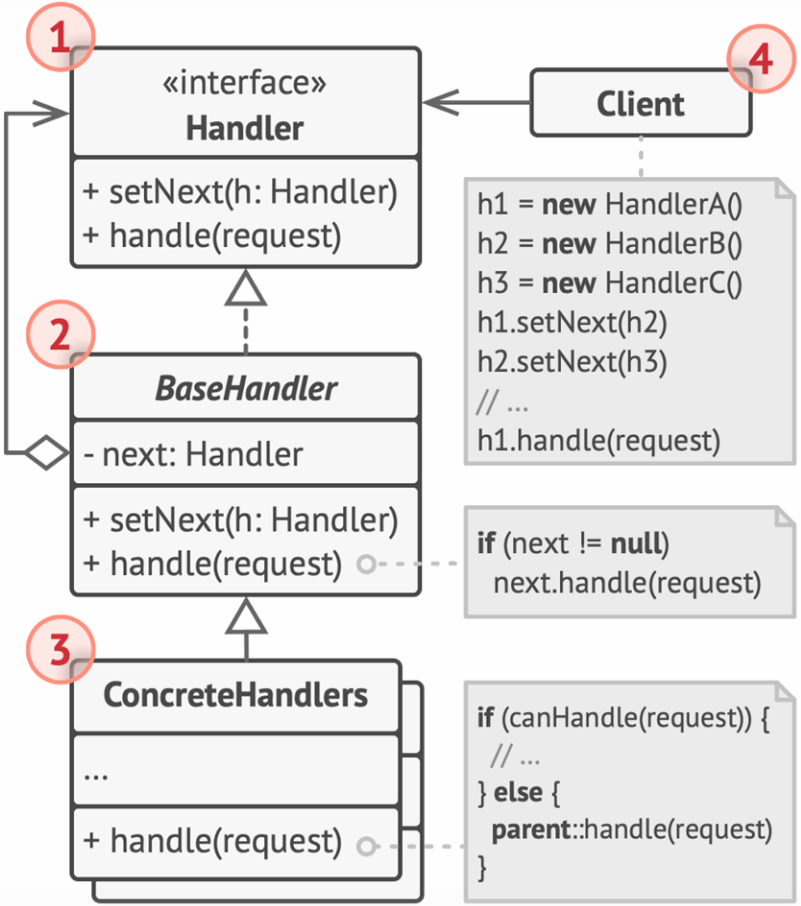
# Chain of Responsibility
# цепочка обязанностей

IThub

Цепочка обязанностей — это поведенческий паттерн проектирования, который позволяет передавать запросы последовательно по цепочке обработчиков.

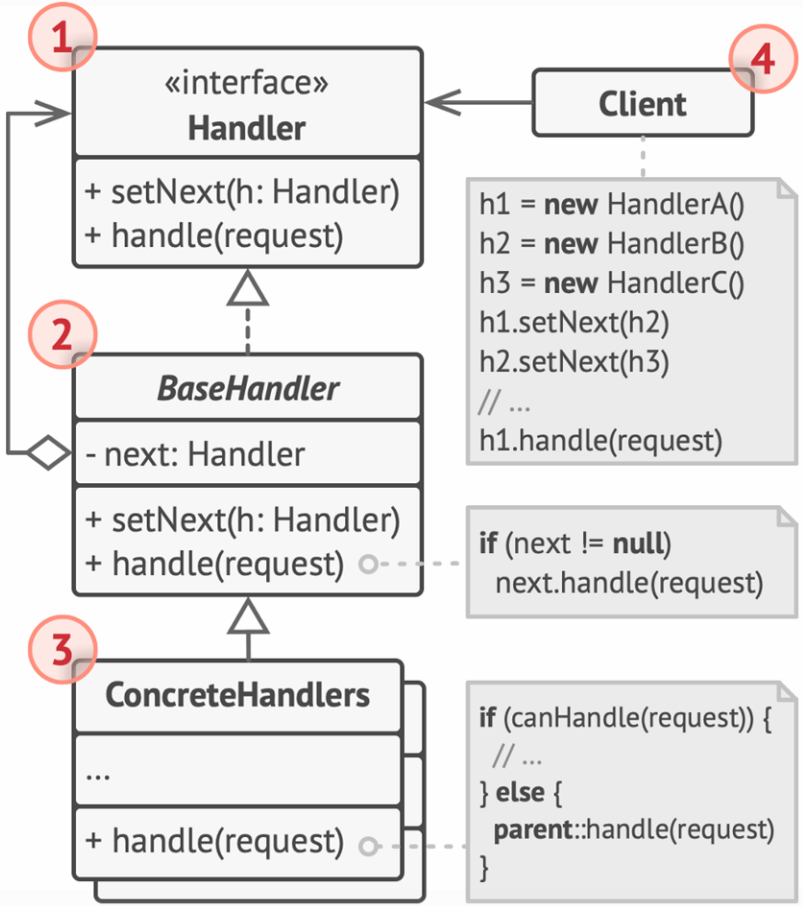
Каждый последующий обработчик решает, может ли он обработать запрос сам и стоит ли передавать запрос дальше по цепи.






1. **Обработчик** определяет общий для всех конкретных обработчиков интерфейс.

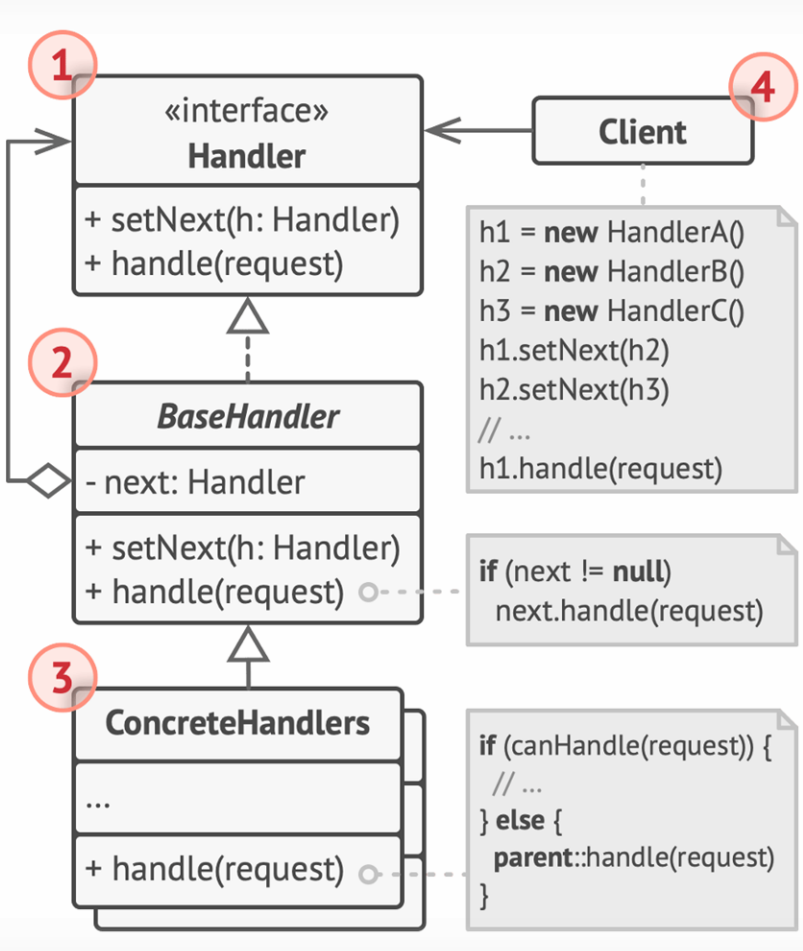
Обычно достаточно описать единственный метод обработки запросов, но иногда здесь может быть объявлен и метод выставления следующего обработчика.




IThub

2. **Базовый обработчик** — опциональный класс, который позволяет избавиться от дублирования одного и того же кода во всех конкретных обработчиках.

Обычно этот класс имеет поле для хранения ссылки на следующий обработчик в цепочке.




3. **Конкретные обработчики** содержат код обработки запросов.

При получении запроса каждый обработчик решает, может ли он обработать запрос, а также стоит ли передать его следующему объекту.

4. **Клиент** может либо сформировать цепочку обработчиков единожды, либо перестраивать её динамически, в зависимости от логики программы.

Клиент может отправлять запросы любому из объектов цепочки, не обязательно первому из них.

IThub

# Например

IThub

в RPG персонаж получает урон. Сначала проверяется магический щит, затем броня, а оставшийся урон получает сам герой. Каждый обработчик отвечает за свою часть обработки.

```
interface DamageHandler {
    int handle(int damage);
}
```

```
class Armor implements DamageHandler {
    public int handle(int damage) {
        System.out.println("Броня поглощает 3 урона");
        return Math.max(0, damage - 3);
    }
}
```

```
class MagicShield implements DamageHandler {
    public int handle(int damage) {
        System.out.println("Щит поглощает 5 урона");
        return Math.max(0, damage - 5);
    }
}
```

Где-то в коде:

```
int damage = 20;
damage = new MagicShield().handle(damage);
damage = new Armor().handle(damage);

System.out.println("Получено урона: " + damage);
```

**Преимущество**: обработчики можно добавлять, убирать и менять местами без изменения исходного запроса.

# Chain of Responsibility
# цепочка обязанностей

IThub

✅ уменьшает зависимость между клиентом и обработчиками

✅ реализует принцип единственной обязанности

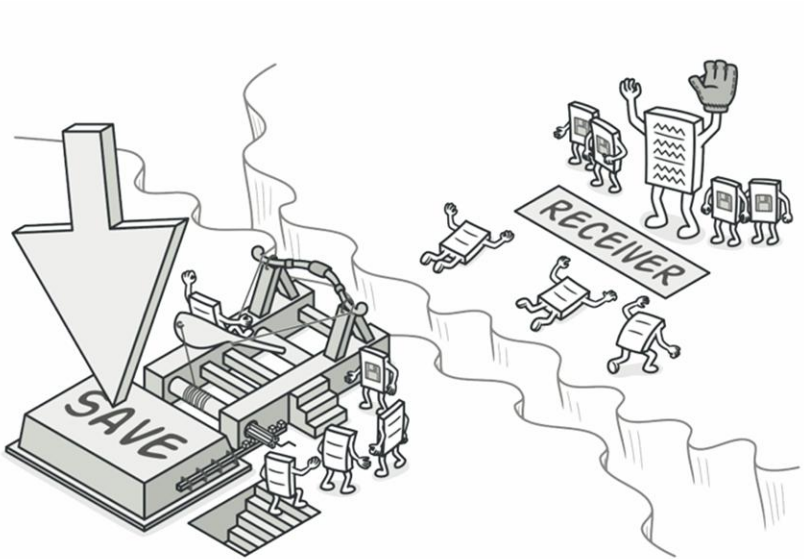
✅ реализует принцип открытости/закрытости

❌ запрос может остаться никем не обработанным

# Command
# команда

IThub

**Команда** — это поведенческий паттерн проектирования, который превращает запросы в объекты, позволяя передавать их как аргументы при вызове методов, ставить запросы в очередь, логировать их, а также поддерживать отмену операций.




**Идея: представить действие в виде отдельного объекта**

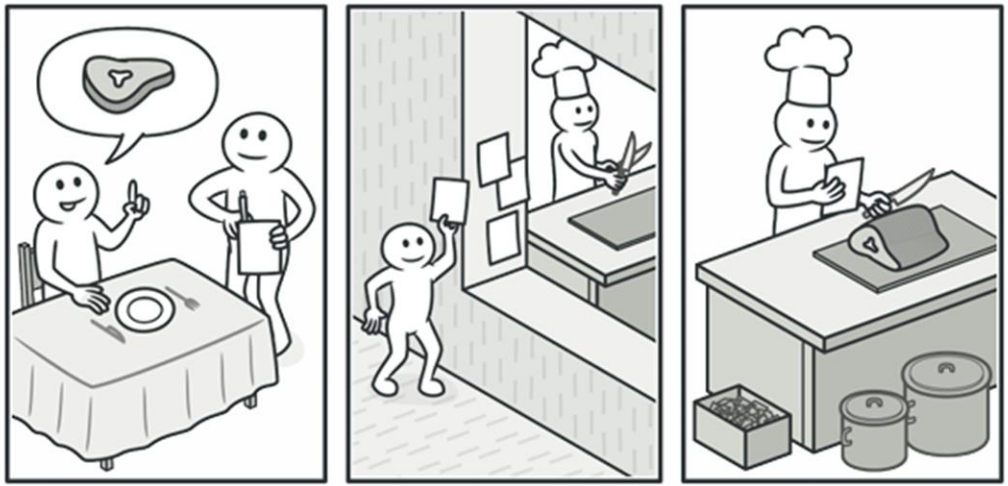
**Вы** заходите в ресторан и садитесь за столик.

К вам подходит вежливый **официант** и принимает заказ, записывая все пожелания в блокнот. Откланявшись, он уходит на кухню, где вырывает лист из блокнота и клеит на стену.

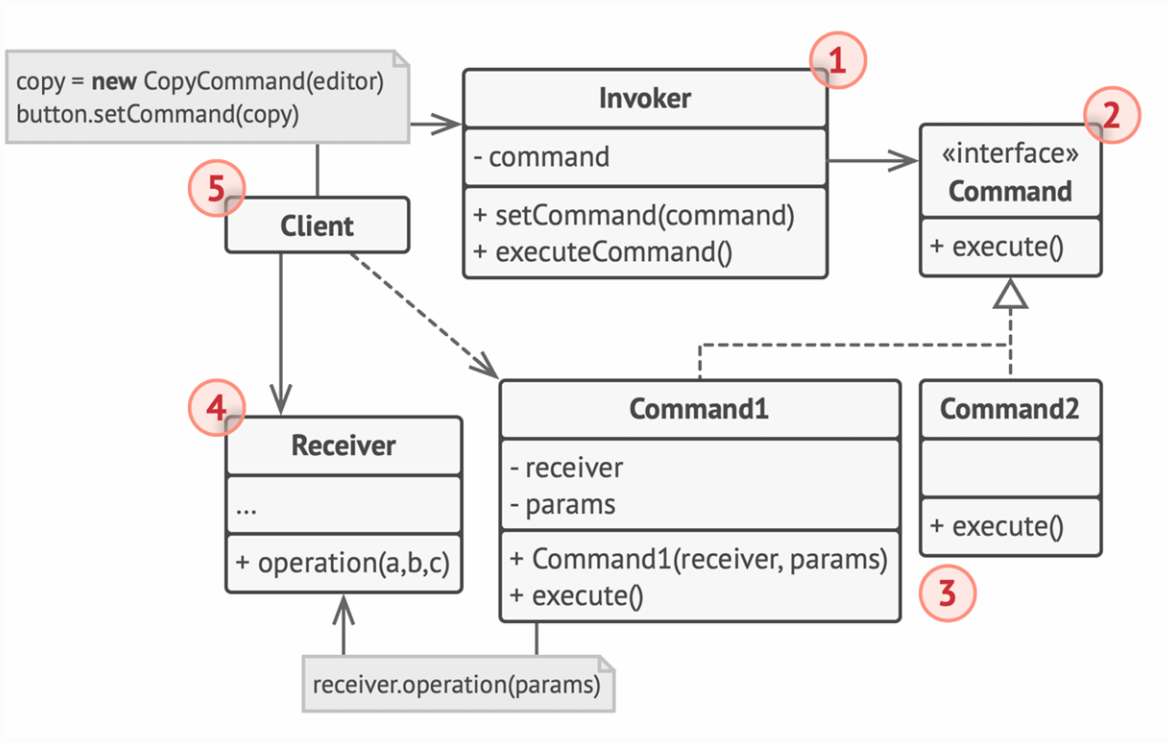
Далее лист оказывается в руках **повара**, который читает содержание заказа и готовит заказанные блюда.

В этом примере вы являетесь отправителем, официант с блокнотом — командой, а повар — получателем. Как и в паттерне, вы не соприкасаетесь напрямую с поваром.

# Пример

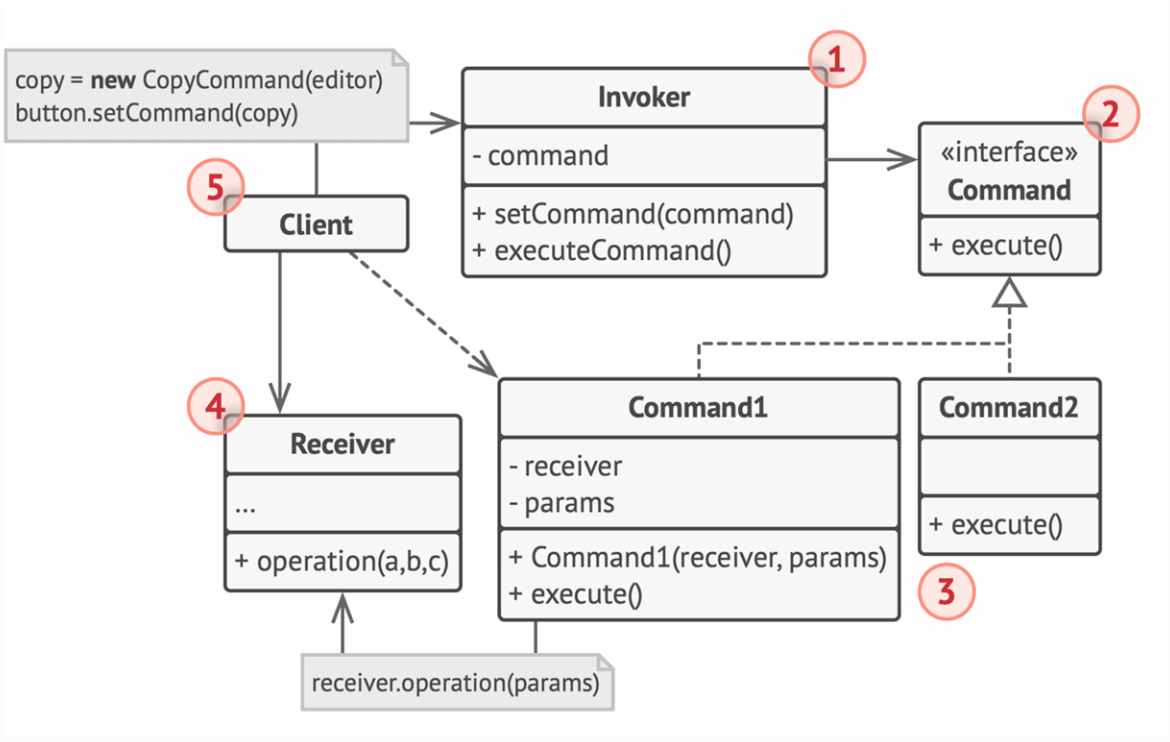


*Пример заказа в ресторане.*




1. **Отправитель** хранит ссылку на объект команды и обращается к нему, когда нужно выполнить какое-то действие.

Отправитель работает с командами только через их общий интерфейс. Он не знает, какую конкретно команду использует, так как получает готовый объект команды от клиента.



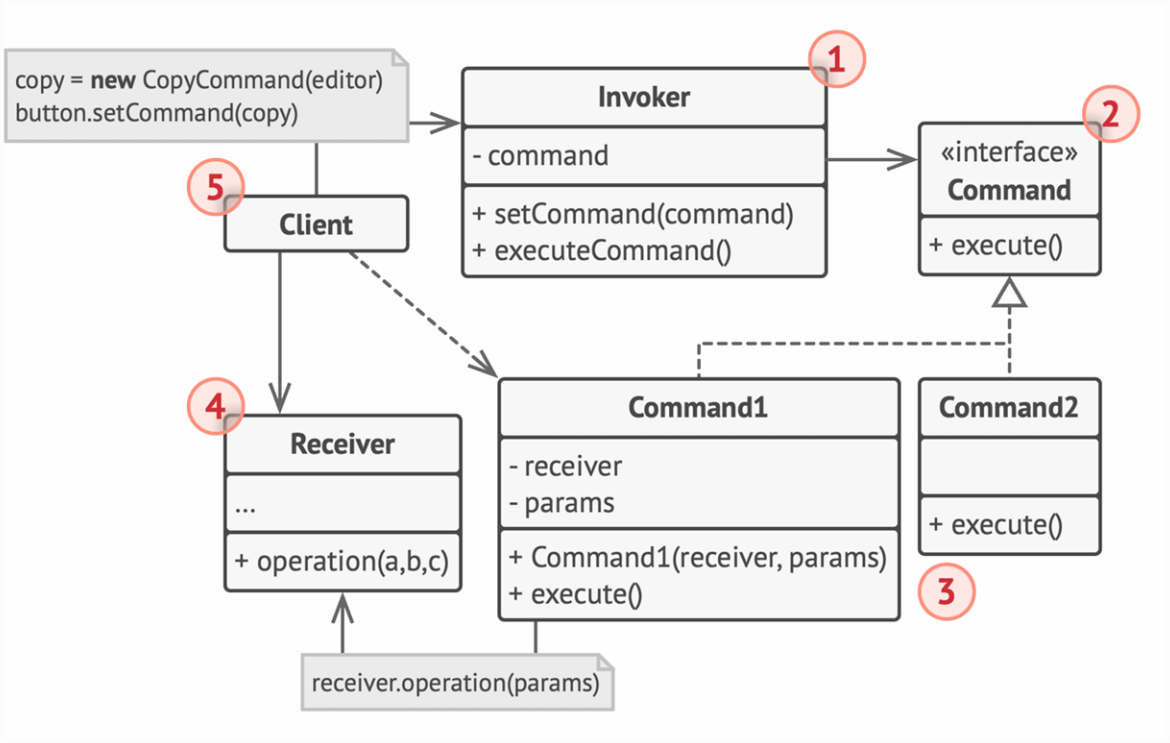

**2. Команда** описывает общий для всех конкретных команд интерфейс.

Обычно здесь описан всего один метод для запуска команды.

**3. Конкретные команды** реализуют различные запросы, следуя общему интерфейсу команд.

Обычно команда не делает всю работу самостоятельно, а лишь передаёт вызов получателю, которым является один из объектов бизнес-логики.

IThub




**4. Получатель** содержит бизнес-логику программы. В этой роли может выступать практически любой объект.

5. **Клиент** создаёт объекты конкретных команд, передавая в них все необходимые параметры, среди которых могут быть и ссылки на объекты получателей.

После этого клиент связывает объекты отправителей с созданными командами.

IThub

В RPG любое действие игрока — атака, лечение, применение заклинания или использование предмета — можно оформить как команду.

```
interface Command {
    void execute();
}
```

```
class Hero {
    void attack() {
        System.out.println("Герой атакует мечом!");
    }
}
```

```
class AttackCommand implements Command {
    private Hero hero;

    AttackCommand(Hero hero) {
        this.hero = hero;
    }

    public void execute() {
        hero.attack();
    }
}
```

**Где-то в коде:**

```
Hero hero = new Hero();
Command command = new AttackCommand(hero);

command.execute();
```

IThub

# Command
# команда

IThub

✅ убирает прямую зависимость между объектами, вызывающими операции, и объектами, которые их непосредственно выполняют.

✅ позволяет реализовать простую отмену и повтор операций

✅ позволяет собирать сложные команды из простых

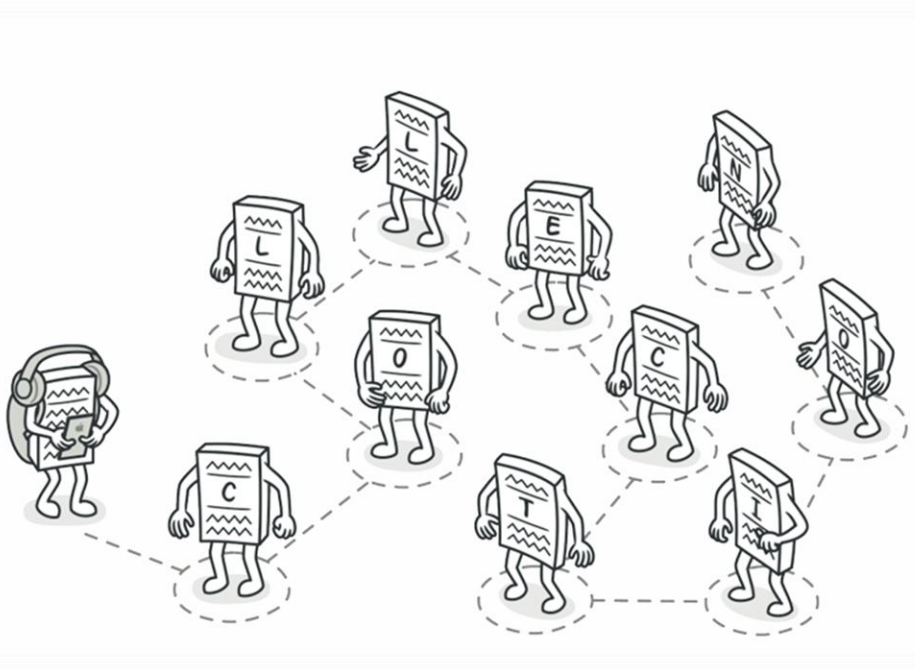
❌ усложняет код программы из-за введения множества дополнительных классов

# Iterator

# итератор

IThub

**Итератор — это поведенческий паттерн проектирования, который даёт возможность последовательно обходить элементы составных объектов, не раскрывая их внутреннего представления.**

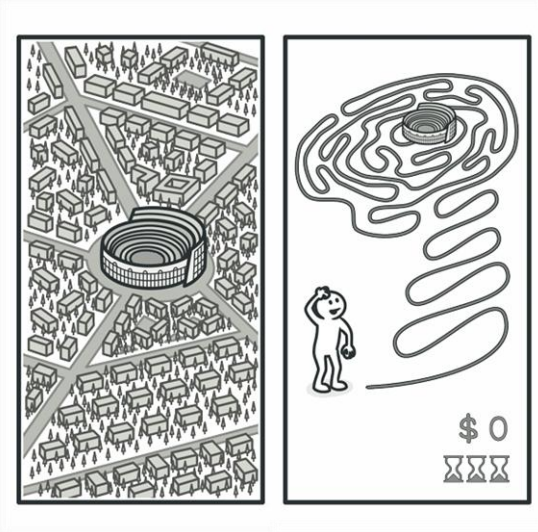


Вы планируете полететь в Рим и обойти все достопримечательности за пару дней:

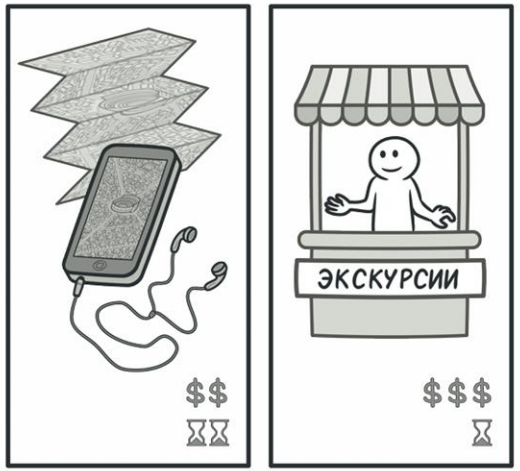
1. Вы можете долго петлять узкими улочками, пытаясь найти Колизей.
2. Вы можете воспользоваться виртуальным гидом, скачанным на телефон.
3. Нанять локального гида.

Таким образом, Рим выступает коллекцией достопримечательностей, а ваш мозг, навигатор или гид — итератором по коллекции.

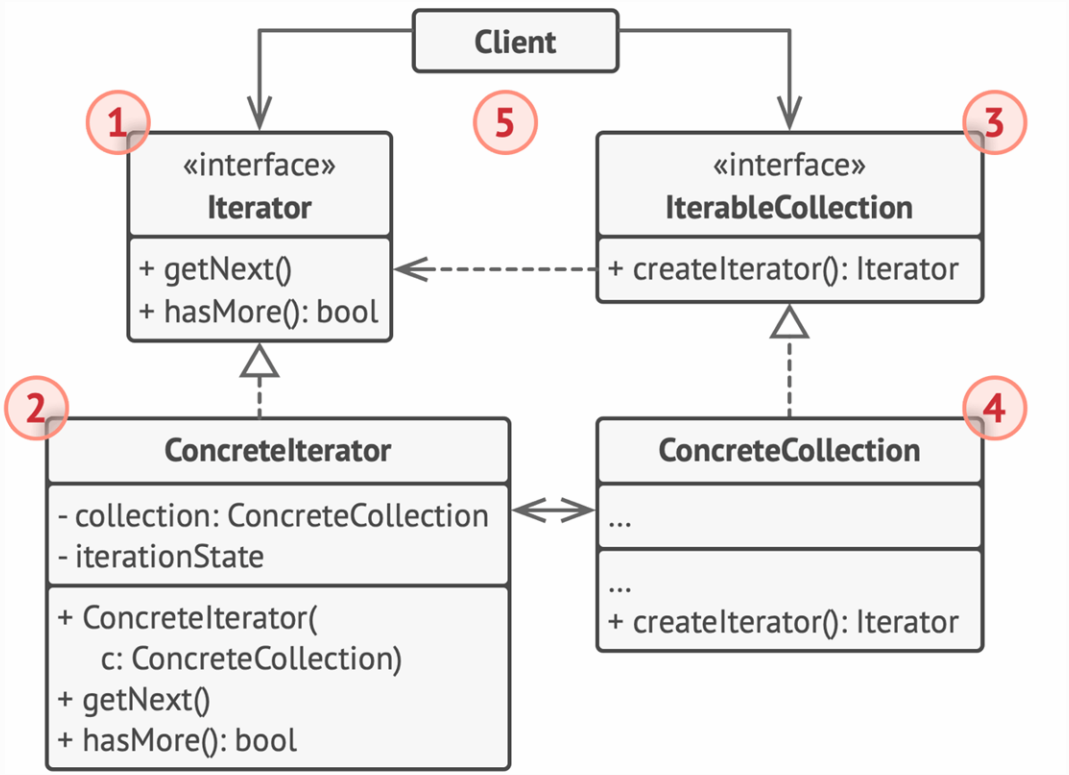
Вы, как клиентский код, можете выбрать один из итераторов, отталкиваясь от решаемой задачи и доступных ресурсов.




IThub



IThub

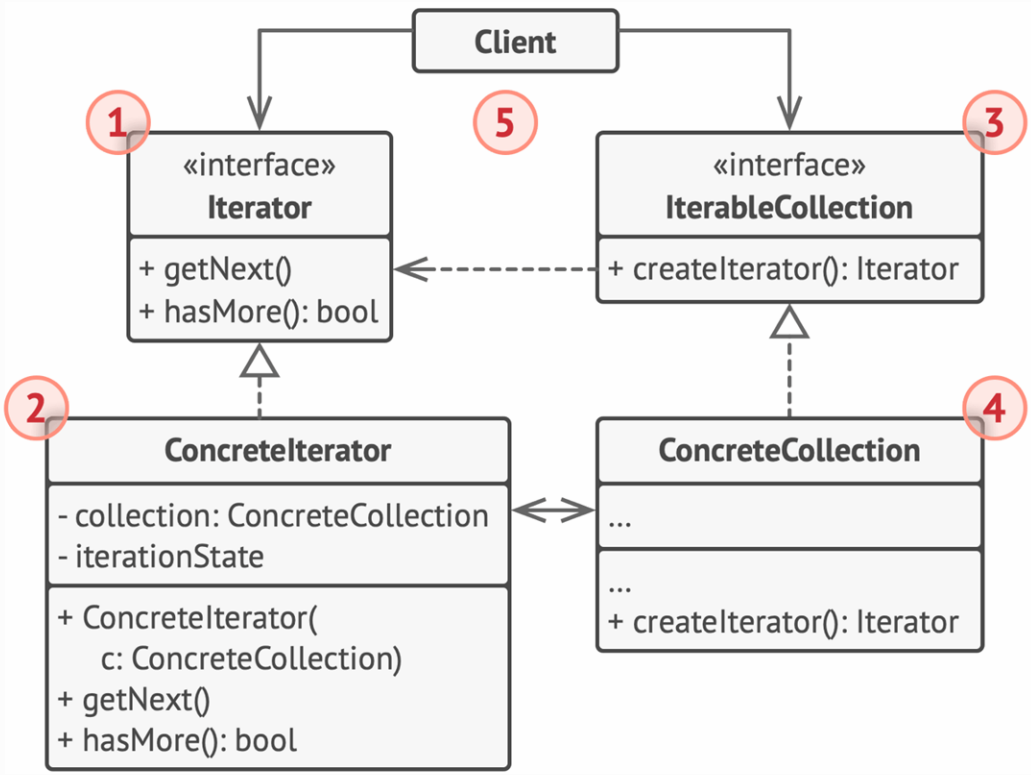



1. **Итератор** описывает интерфейс для доступа и обхода элементов коллекции.

2. **Конкретный итератор** реализует алгоритм обхода какой-то конкретной коллекции.

Объект итератора должен сам отслеживать текущую позицию при обходе коллекции, чтобы отдельные итераторы могли обходить одну и ту же коллекцию независимо

IThub



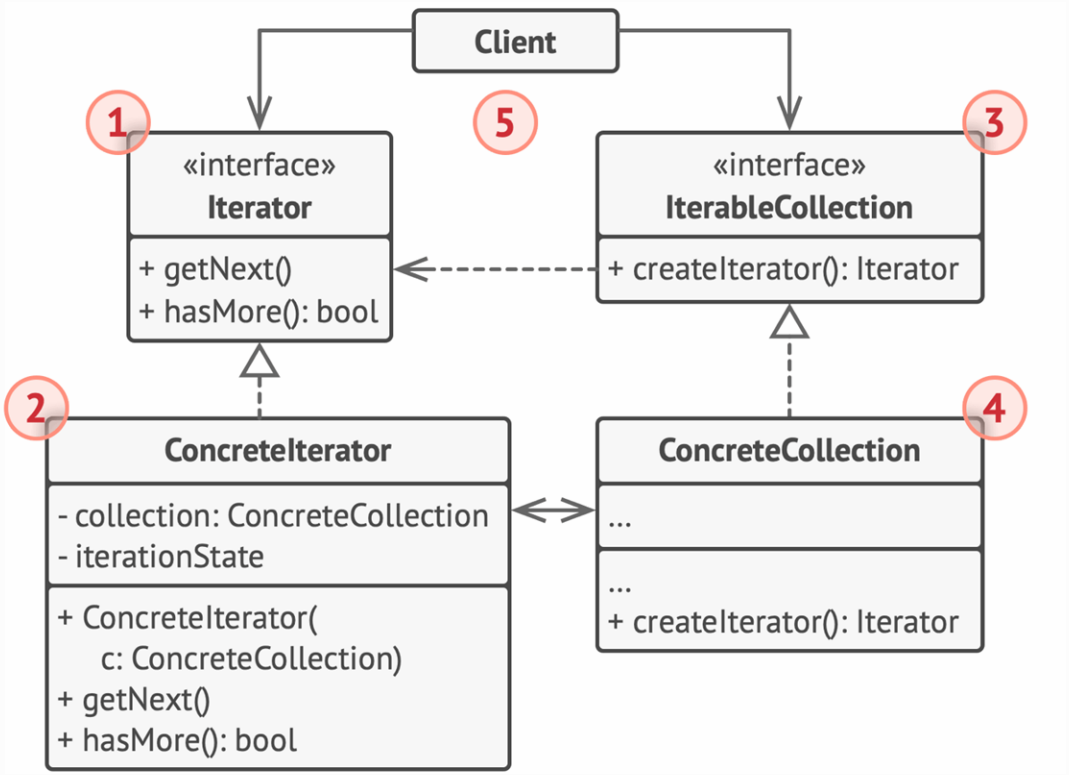

3. **Коллекция** описывает интерфейс получения итератора из коллекции.

Коллекции не всегда являются списком. Это может быть и база данных и удалённое API.

4. **Конкретная коллекция** возвращает новый экземпляр определённого конкретного итератора, связав его с текущим объектом коллекции.

Возвращает интерфейс итератора, что позволяет клиенту не зависеть от конкретных классов итераторов.

IThub




5. **Клиент** работает со всеми объектами через интерфейсы коллекции и итератора.

Так клиентский код не зависит от конкретных классов, что позволяет применять различные итераторы, не изменяя существующий код программы.

В RPG герой может иметь инвентарь с оружием, зельями и броней.
Итератор позволяет пройти по всем предметам и вывести их названия.

```
abstract class Item {
    private String name;
    private int value;

    //  геттеры, сеттеры, конструктор

    public abstract String getType();
}
```

```
// оружие
class Weapon extends Item {
    private int damage;

    // конструктор, геттеры, сеттеры

    @Override
    public String getType() {
        return "Оружие";
    }
}
```

```
// зелье
class Potion extends Item {
    private int healing;

    // конструктор, геттеры, сеттеры

@Override
    public String getType() {
        return "Зелье";
    }
}
```

```
// броня
class Armor extends Item {
    private int defense;

    // конструктор, геттеры, сеттеры

    @Override
    public String getType() {
        return "Броня";
    }
}
```

IThub

```
abstract class AbstractIterator<T> {
    public abstract boolean hasNext();

    public abstract T next();
}
```

- hasNext() проверяет, остались ли предметы
- next() возвращает следующий предмет и перемещает итератор вперёд.

```
class InventoryIterator extends AbstractIterator<Item> {
    private List<Item> items;
    private int position = 0;

    public InventoryIterator(List<Item> items) {
        this.items = items;
    }

    @Override
    public boolean hasNext() {
        return position < items.size();
    }

    @Override
    public Item next() {
        if (!hasNext()) {
            throw new NoSuchElementException(
                    "В инвентаре больше нет предметов");
        }

        return items.get(position++);
    }
}
```

- Конкретный итератор хранит ссылку на список предметов и текущую позицию.
- переменная position указывает на следующий предмет. После вызова next() позиция увеличивается на единицу.

IThub

```
class Inventory {
    private List<Item> items = new ArrayList<>();

    public void addItem(Item item) {
        items.add(item);
    }

    public AbstractIterator<Item> createIterator() {
        return new InventoryIterator(items);
    }

    public int size() {
        return items.size();
    }
}
```

Класс Inventory хранит предметы, добавляет новые и создаёт итератор. Код, использующий инвентарь, не обязан самостоятельно управлять индексами.

IThub

# Использование итератора:

```
public class Main {
    public static void main(String[] args) {
        Inventory inventory = new Inventory();

        inventory.addItem(new Weapon("Стальной меч", 100, 25));
        inventory.addItem(new Potion("Зелье лечения", 30, 50));
        inventory.addItem(new Armor("Железный нагрудник", 150, 12));

        AbstractIterator<Item> iterator = inventory.createIterator();

        System.out.println("Инвентарь героя:");

        while (iterator.hasNext()) {
            Item item = iterator.next();

            System.out.println(
                item.getType() + ": " + item.getName()
                    + ", цена: " + item.getValue() + " золота"
            );
        }
    }
}
```

# Iterator
# итератор

IThub

✅ упрощает классы хранения данных

✅ позволяет реализовать различные способы обхода структуры данных

❌ не оправдан, если можно обойтись простым циклом

# Interpreter
# интерпретатор

IThub

Он определяет грамматику простого предметного языка и создает механизм для интерпретации предложений этого языка.

Идея: определить простые правила языка и научить программу интерпретировать выражения на этом языке.

IThub

# Например

в RPG можно создать небольшой язык игровых условий для квестов.

Например, условие level >= 5 означает, что герой должен достичь пятого уровня.

```
interface Expression {
    boolean interpret(int level);
}
```

```
class MinLevel implements Expression {
    private int requiredLevel;

    MinLevel(int requiredLevel) {
        this.requiredLevel = requiredLevel;
    }

    public boolean interpret(int level) {
        return level >= requiredLevel;
    }
}
```

Где-то в коде:

```
Expression questCondition = new MinLevel(5);
System.out.println(questCondition.interpret(7));
// true — квест доступен
```

**Преимущество**: игровые условия можно описывать через выражения, не создавая отдельный класс для каждого конкретного квеста.

# Interpreter
# интерпретатор

IThub

✅ легко добавлять новые правила и выражения

✅ можно создавать собственный язык команд или условий

❌ каждое правило может требовать отдельной реализации (много классов)

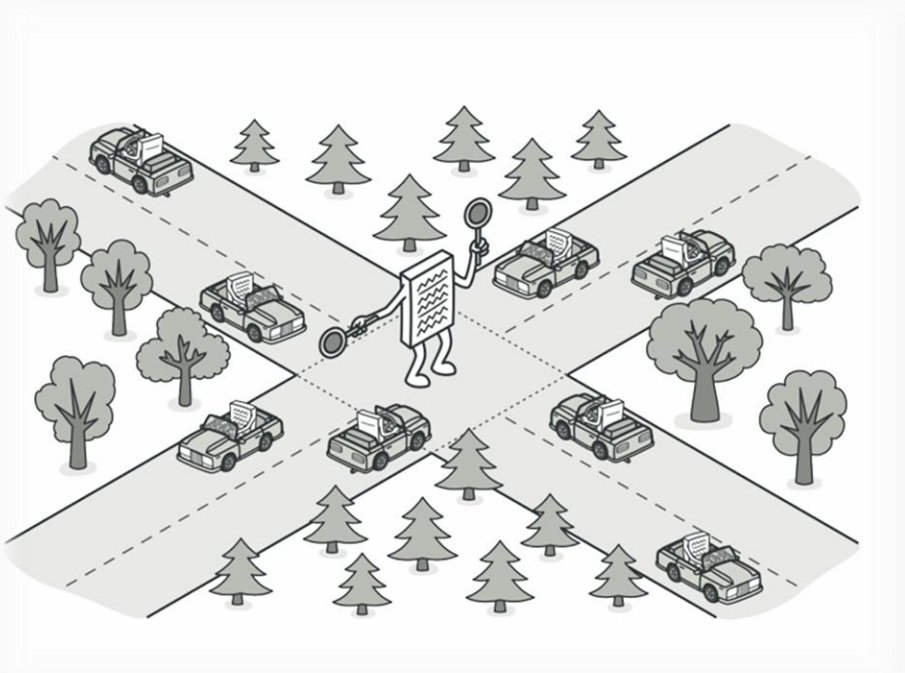
❌ для больших и сложных языков структура становится громоздкой

# Mediator

# посредник

IThub

Посредник — это поведенческий паттерн проектирования, который позволяет уменьшить связанность множества классов между собой, благодаря перемещению этих связей в один класс-посредник.



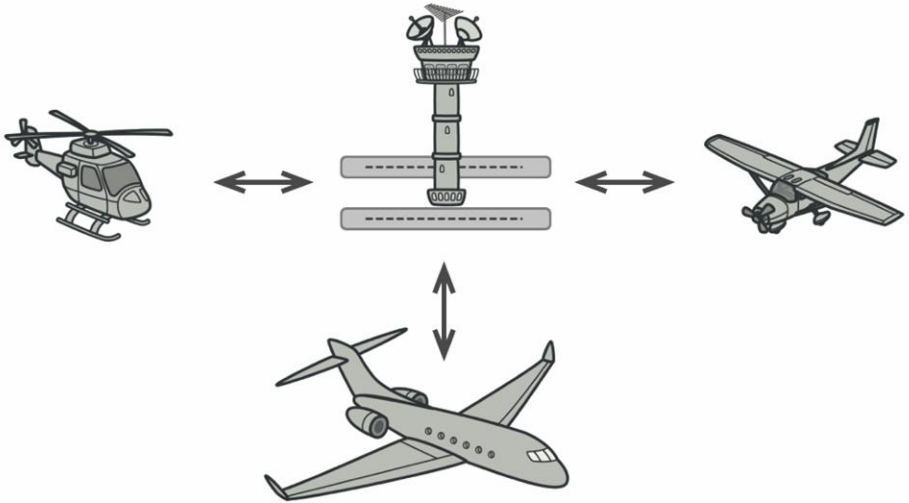
Идея: организовать взаимодействие объектов через специальный объект-посредник.

# Пример

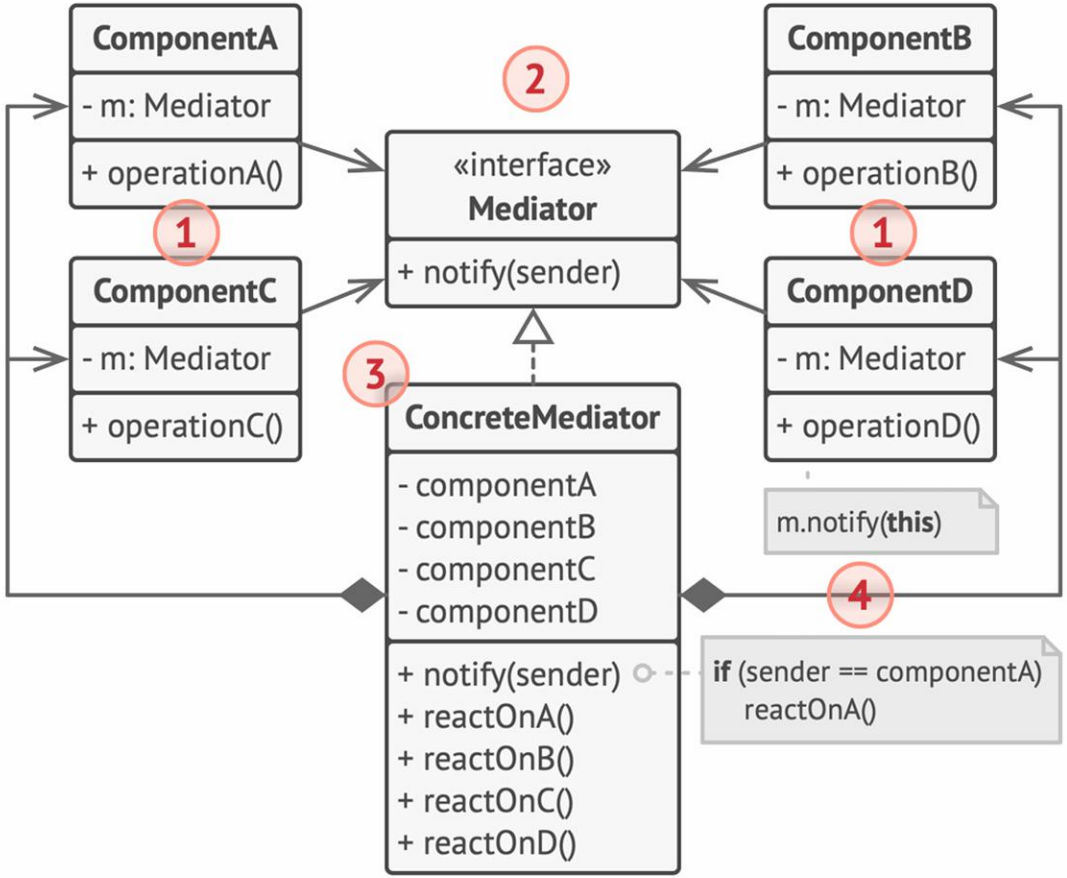
IThub

Пилоты садящихся или улетающих самолётов не общаются напрямую с другими пилотами.

Вместо этого они связываются с диспетчером, который координирует действия нескольких самолётов одновременно.



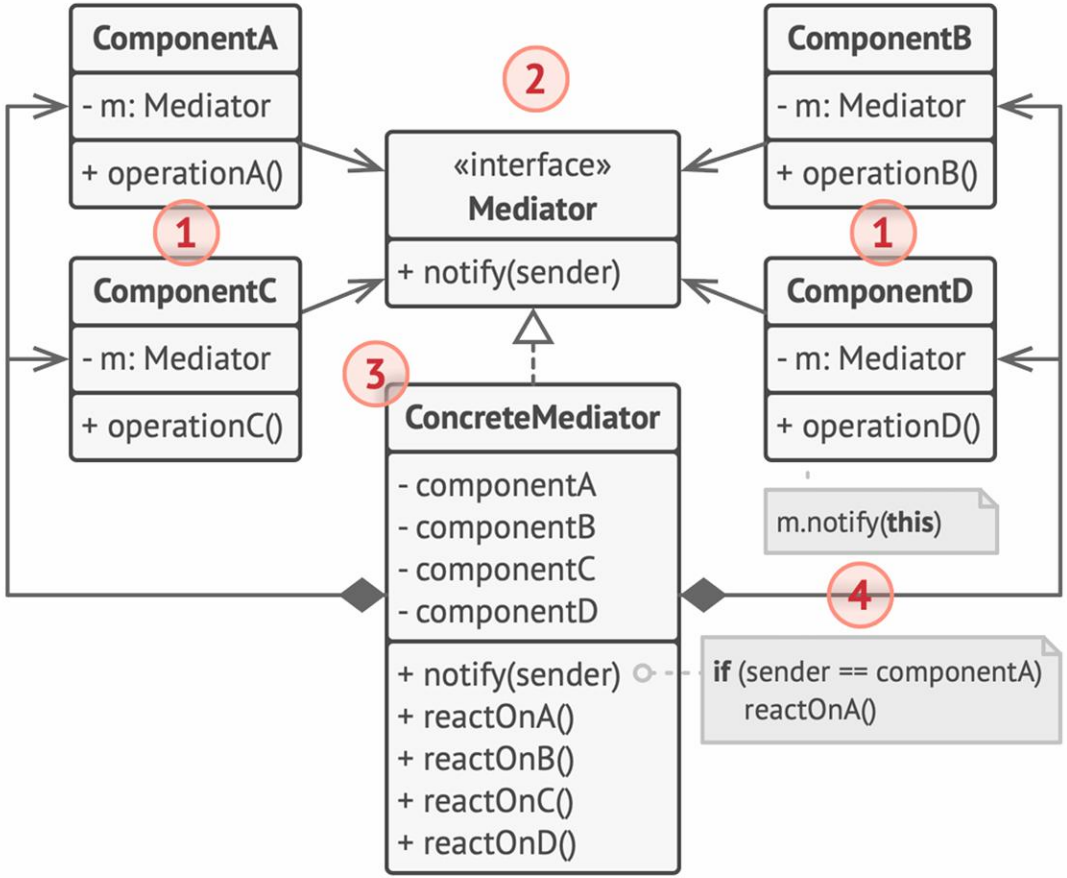
*Пилоты самолётов общаются не напрямую, а через диспетчера.*




1. **Компоненты** — это разнородные объекты, содержащие бизнес-логику программы.

Каждый компонент хранит ссылку на объект посредника, но работает с ним только через абстрактный интерфейс посредников.

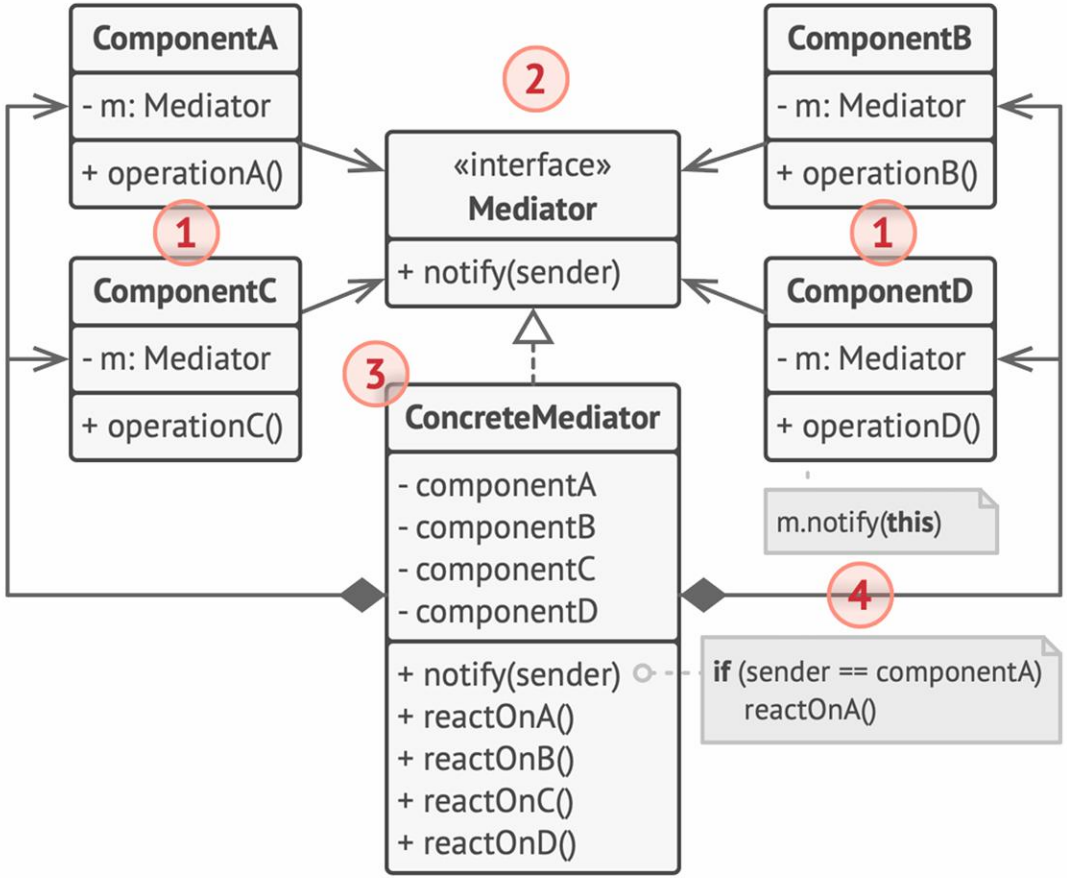
Благодаря этому, компоненты можно повторно использовать в другой программе, связав их с посредником другого типа.




2. **Посредник** определяет интерфейс для обмена информацией с компонентами.

Обычно хватает одного метода, чтобы оповещать посредника о событиях, произошедших в компонентах.

В параметрах этого метода можно передавать детали события: ссылку на компонент, в котором оно произошло, и любые другие данные




3. **Конкретный посредник** содержит код взаимодействия нескольких компонентов между собой.

4. Компоненты не должны общаться друг с другом напрямую.

Если в компоненте происходит важное событие, он должен оповестить своего посредника, а тот сам решит — касается ли событие других компонентов, и стоит ли их оповещать.

```
class BattleMediator {
    void attack(String attacker, String target) {
        System.out.println(
            attacker + " атакует " + target
        );
    }
}
```

```
class Warrior {
    private BattleMediator mediator;

    Warrior(BattleMediator mediator) {
        this.mediator = mediator;
    }

    void attackEnemy(String enemy) {
        mediator.attack("Воин", enemy);
    }
}
```

В RPG несколько персонажей могут участвовать в бою.
Если каждый персонаж напрямую управляет остальными, код становится сложным.
Посредник централизует взаимодействие.

```
BattleMediator battle = new BattleMediator();
Warrior warrior = new Warrior(battle);
warrior.attackEnemy("Орк");
```

Персонажам не нужно знать внутреннее устройство всех остальных участников боя.
Они взаимодействуют через посредника.

# Mediator

## посредник

IThub

✅ устраняет зависимости между компонентами, позволяя повторно их использовать

✅ упрощает взаимодействие между компонентами

✅ централизует управление в одном месте

❌ Посредник может сильно раздуться

# Memento

## снимок или хранитель

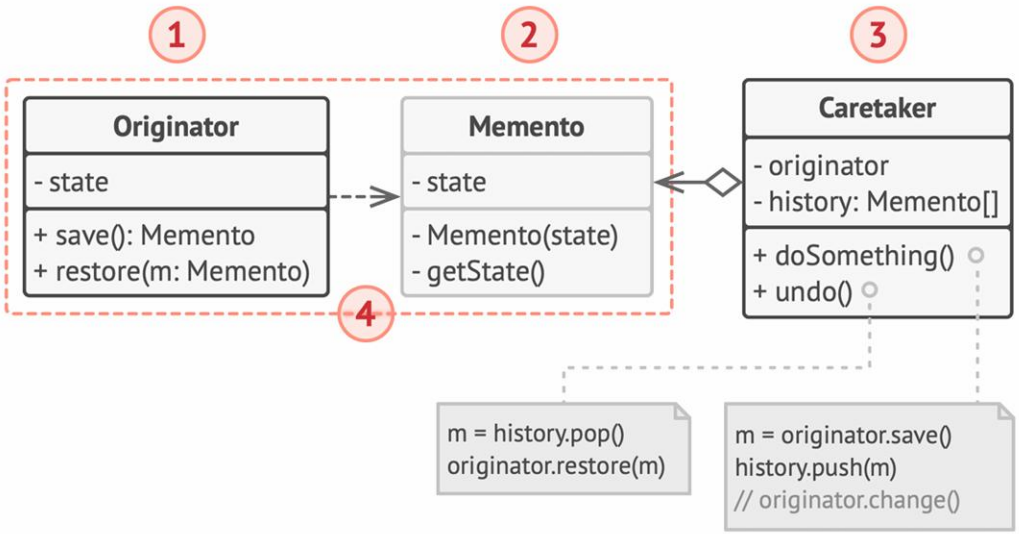
IThub

**Снимок** — это поведенческий паттерн проектирования, который позволяет сохранять и восстанавливать прошлые состояния объектов, не раскрывая подробностей их реализации.

**Идея**: сохранить состояние объекта, чтобы впоследствии восстановить его.



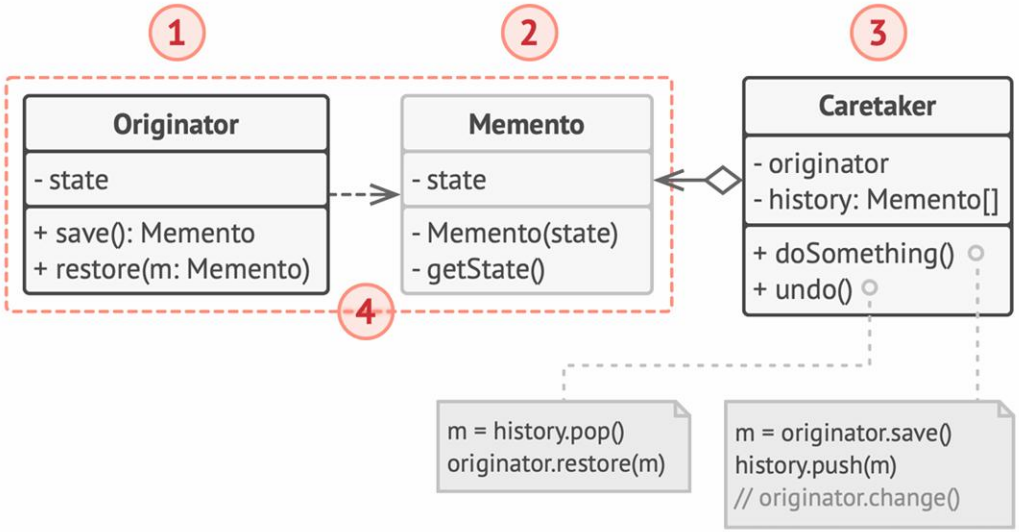
**Классическая реализация** паттерна полагается на механизм вложенных классов, который доступен лишь в некоторых языках программирования (C++, C#, Java).




1. **Создатель** может производить снимки своего состояния, а также воспроизводить прошлое состояние, если подать в него готовый снимок.

2. **Снимок** — это простой объект данных, содержащий состояние создателя.

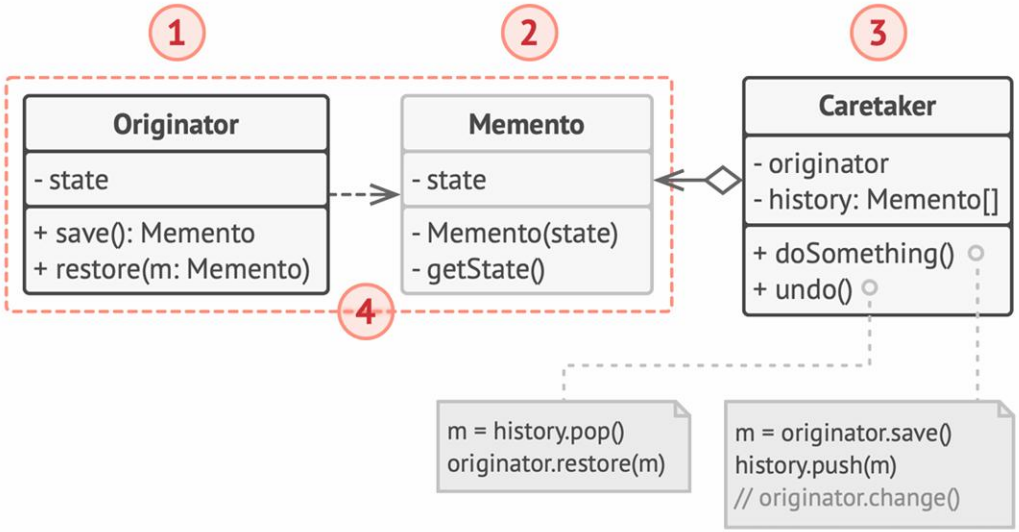
Надёжнее всего сделать объекты снимков неизменяемыми, передавая в них состояние только через конструктор.




3. **Опекун** должен знать, когда делать снимок создателя и когда его нужно восстанавливать.

Опекун может хранить историю прошлых состояний создателя в виде стека из снимков.

Когда понадобится отменить выполненную операцию, он возьмёт «верхний» снимок из стека и передаст его создателю для восстановления

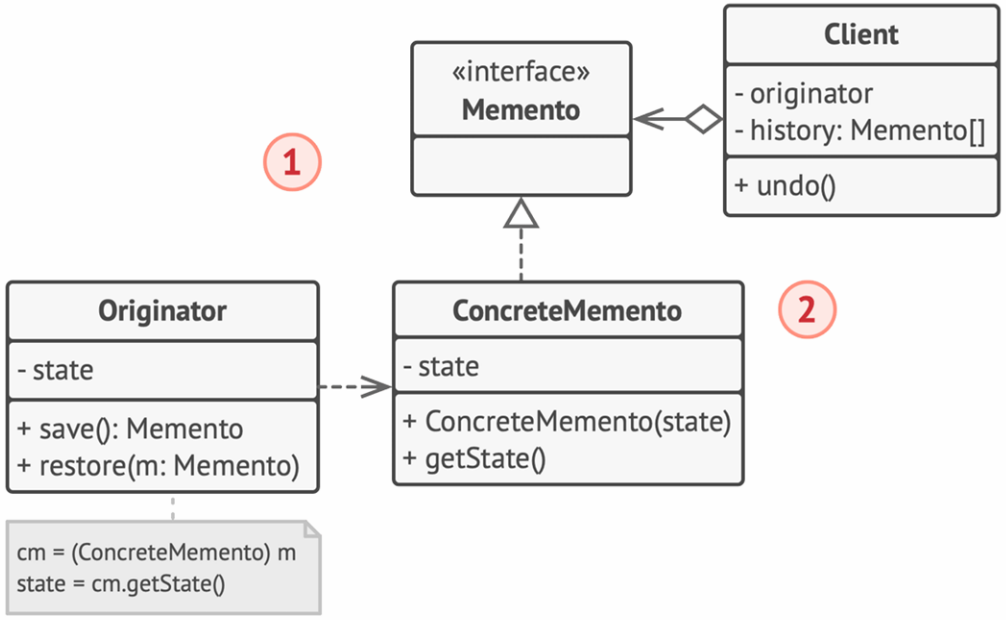



В данной реализации снимок — это внутренний класс по отношению к классу создателя.

Именно поэтому он имеет полный доступ к полям и методам создателя, даже приватным.

С другой стороны, опекун не имеет доступа ни к состоянию, ни к методам снимков и может всего лишь хранить ссылки на эти объекты.

**Реализация с пустым промежуточным интерфейсом** паттерна подходит для языков, не имеющих механизма вложенных классов (например, PHP).




1. В этой реализации **создатель** работает напрямую с **конкретным классом** снимка, а **опекун** — только с его ограниченным **интерфейсом**.

2. Благодаря этому достигается тот же эффект, что и в классической реализации.

Создатель имеет полный доступ к снимку, а опекун — нет.

В RPG паттерн Memento можно использовать для сохранения состояния героя перед боем или для реализации отмены последнего действия.

- **Originator** (Hero) — объект, состояние которого нужно сохранить и восстановить.
- **Memento** (HeroMemento) — отдельный объект, содержащий снимок состояния героя.
- **Caretaker** (SaveManager) — хранитель снимков, который сохраняет их и позволяет получить сохранение, не изменяя его содержимое.

IThub

```
class Hero {
    private int health;
    private int mana;
    private int level;

    // конструктор

    public void takeDamage(int damage) {
        health = Math.max(0, health - damage);
    }

    public void useMana(int amount) {
        mana = Math.max(0, mana - amount);
    }

    // создание снимка состояния
    public HeroMemento save() {
        return new HeroMemento(health, mana, level);
    }

    // восстановление состояния
    public void restore(HeroMemento memento) {
        this.health = memento.getHealth();
        this.mana = memento.getMana();
        this.level = memento.getLevel();
    }
}
```

Герой умеет создавать снимок своего состояния и восстанавливать его.

IThub

```
final class HeroMemento {
    private final int health;
    private final int mana;
    private final int level;

    public HeroMemento(int health, int mana, int level) {
        this.health = health;
        this.mana = mana;
        this.level = level;
    }

    public int getHealth() {
        return health;
    }

    public int getMana() {
        return mana;
    }

    public int getLevel() {
        return level;
    }
}
```

Этот отдельный объект **HeroMemento** хранит значения полей героя на момент сохранения.

После создания снимка эти значения нельзя изменить.

**!** Лучше по классической реализации сделать класс HeroMemento статическим вложенным классом.

IThub

```
class SaveManager {
    private List<HeroMemento> saves = new ArrayList<>();

    public void addSave(HeroMemento memento) {
        saves.add(memento);
    }

    public HeroMemento getSave(int index) {
        return saves.get(index);
    }
}
```

SaveManager — хранитель сохранений.

Он отвечает за хранение снимков и не меняет состояние героя самостоятельно.

Если HeroMemento – вложенный класс:

```
class SaveManager {
    private List<Hero.HeroMemento> saves =  new ArrayList<>();

    public void addSave(Hero.HeroMemento memento) {
        saves.add(memento);
    }
}
```

IThub

Где-то в коде

```
public static void main(String[] args) {
        Hero hero = new Hero(100, 80, 5);
        SaveManager saveManager = new SaveManager();

        System.out.println("Перед боем:");
        System.out.println(hero);

        // Сохраняем состояние
        saveManager.addSave(hero.save());

        // Герой сражается с драконом
        hero.takeDamage(70);
        hero.useMana(30);

        System.out.println("\nПосле боя:");
        System.out.println(hero);

        // Загружаем сохранение
        hero.restore(saveManager.getSave(0));

        System.out.println("\nПосле загрузки сохранения:");
        System.out.println(hero);
    }
```

IThub

# Memento

## снимок или хранитель

IThub

✅ не нарушает инкапсуляции исходного объекта

✅ упрощает структуру исходного объекта. Ему не нужно хранить историю версий своего состояния.

❌ требует много памяти, если клиенты слишком часто создают снимки

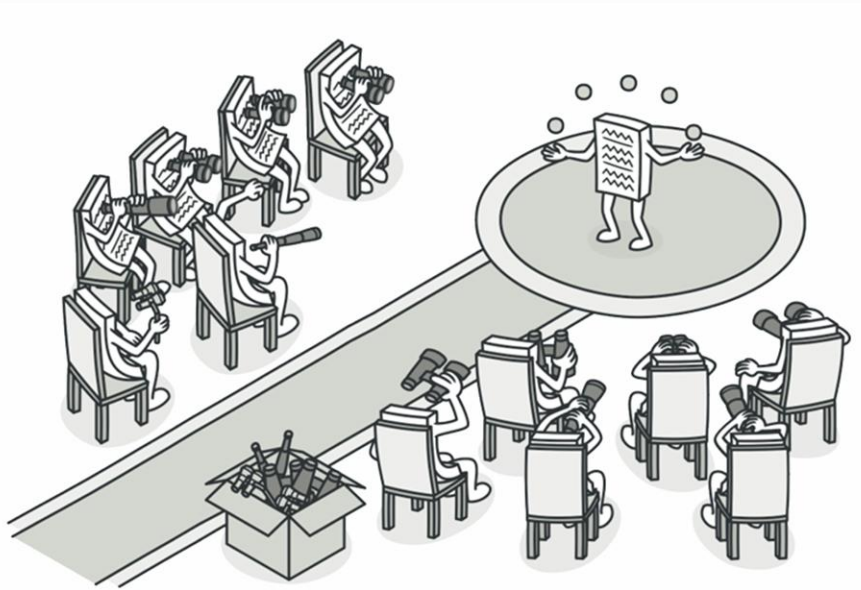
❌ может повлечь дополнительные издержки памяти, если объекты, хранящие историю, не освобождают ресурсы, занятые устаревшими снимками

# Observer
# наблюдатель

IThub

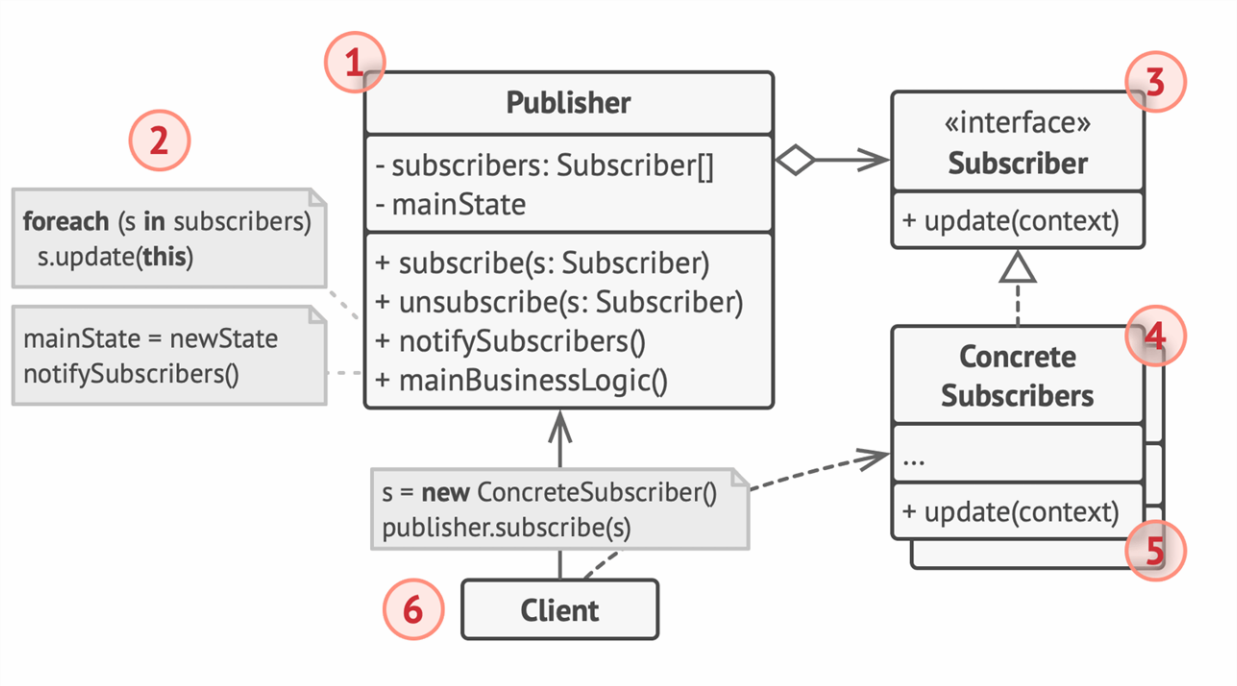
Наблюдатель — это поведенческий паттерн проектирования, который создаёт механизм подписки, позволяющий одним объектам следить и реагировать на события, происходящие в других объектах.

Идея: уведомлять подписчиков об изменениях состояния объекта.



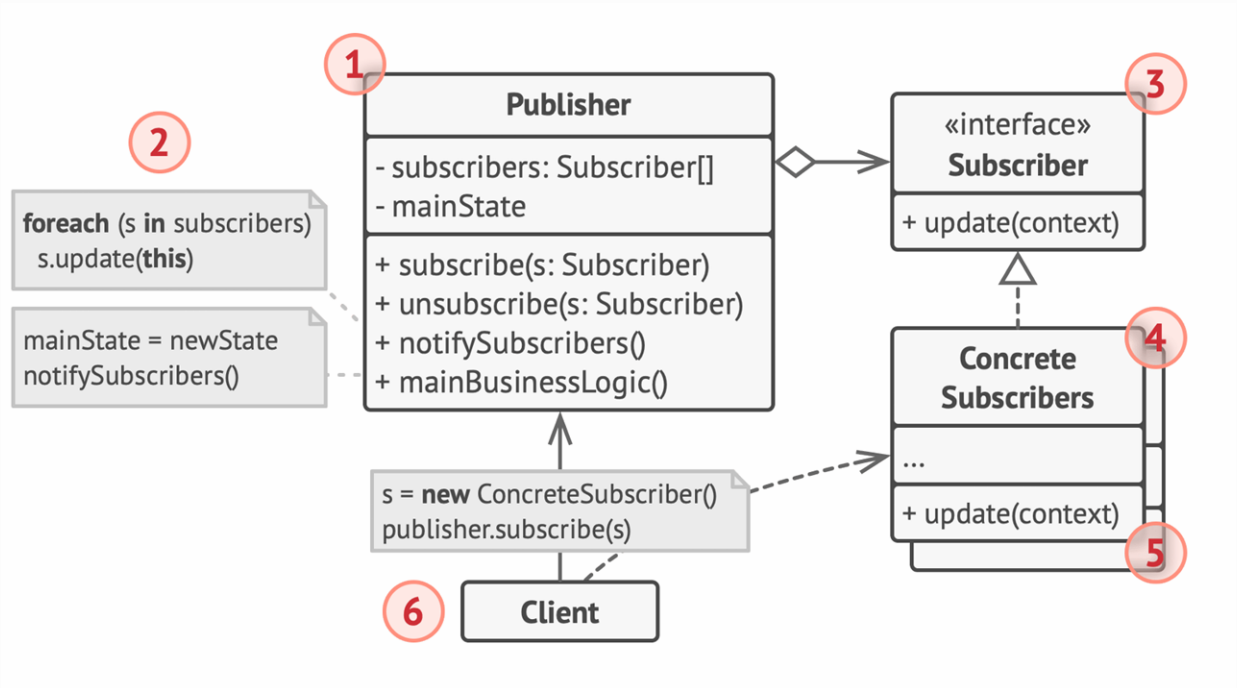
1. **Издатель** владеет внутренним состоянием, изменение которого интересно отслеживать подписчикам.

Издатель содержит механизм подписки: список подписчиков и методы подписки/отписки.



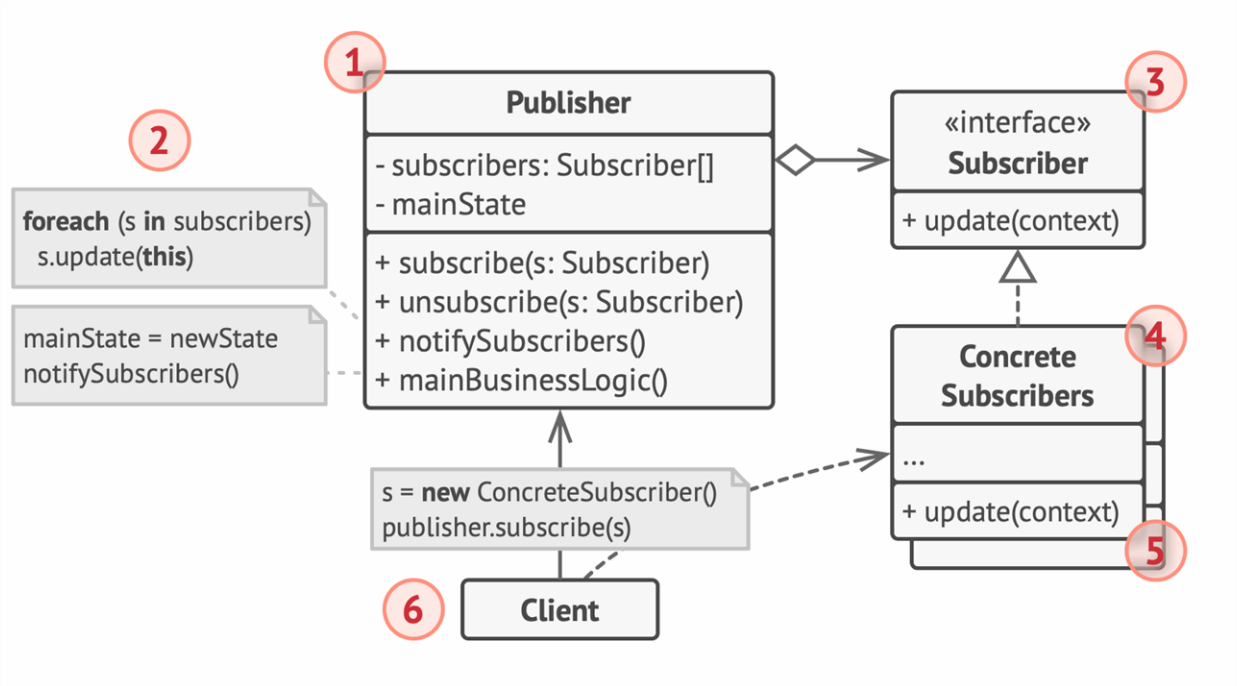
2. Когда внутреннее состояние издателя меняется, он оповещает своих подписчиков.

Для этого издатель проходит по списку подписчиков и вызывает их метод оповещения, заданный в общем интерфейсе подписчиков.



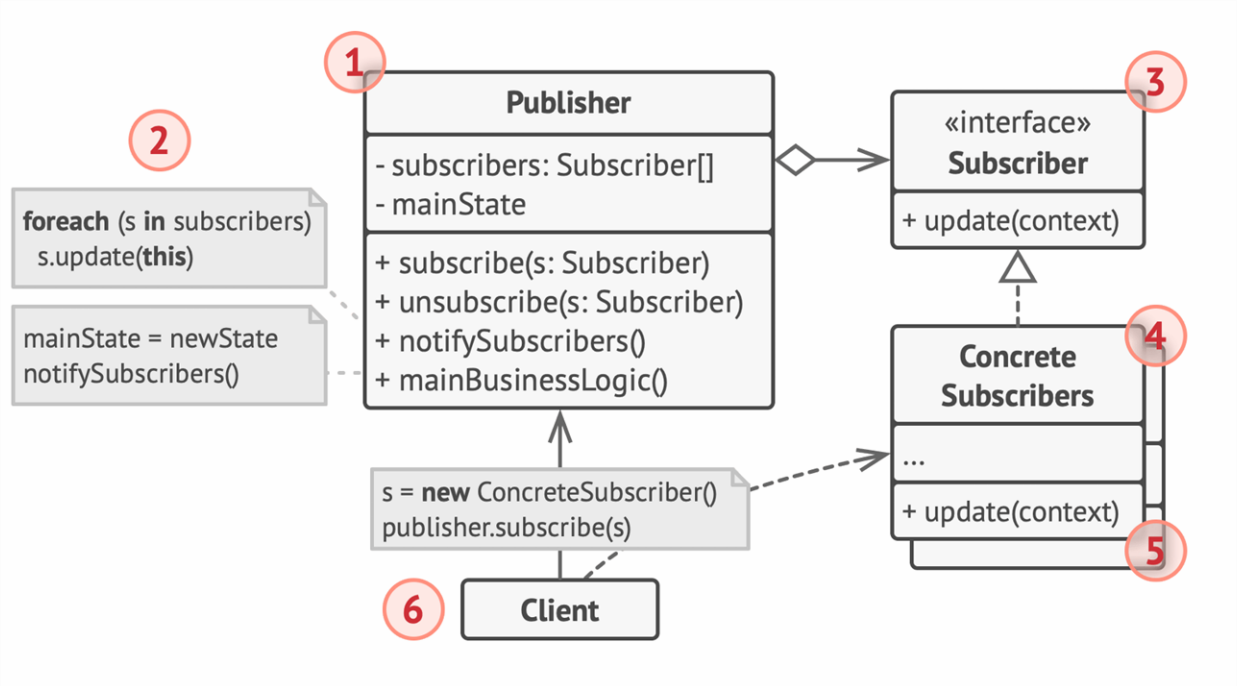
3. **Подписчик** определяет интерфейс, которым пользуется издатель для отправки оповещения.

В большинстве случаев для этого достаточно единственного метода.



4. **Конкретные подписчики** выполняют что-то в ответ на оповещение, пришедшее от издателя.

Эти классы должны следовать общему интерфейсу подписчиков, чтобы издатель не зависел от конкретных классов подписчиков.



5. По приходу оповещения подписчику нужно получить обновлённое состояние издателя.

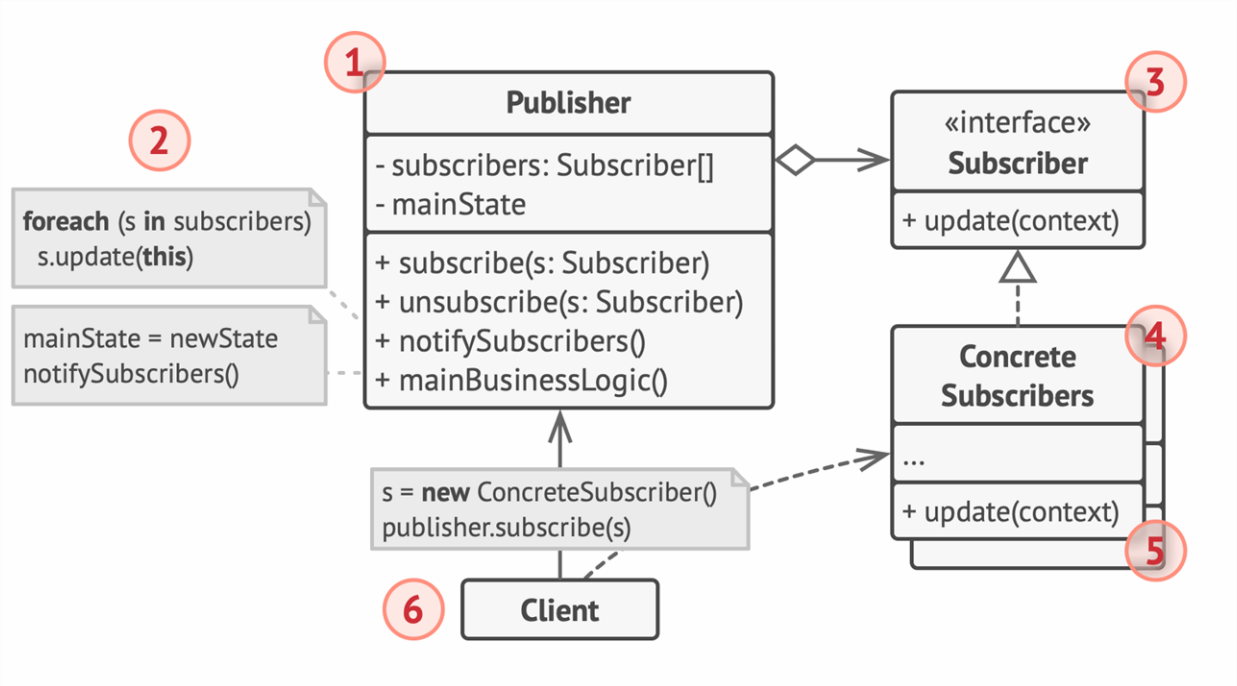
Издатель может передать это состояние через параметры метода оповещения.

Более гибкий вариант — передавать через параметры весь объект издателя, чтобы подписчик мог сам получить требуемые данные.

Как вариант, подписчик может постоянно хранить ссылку на объект издателя, переданный ему в конструкторе.

IThub

6. **Клиент** создаёт объекты издателей и подписчиков, а затем регистрирует подписчиков на обновления в издателях.



# Например

IThub

в RPG, когда здоровье героя уменьшается, интерфейс должен обновить полоску здоровья, а система достижений — проверить условия получения награды.

```
interface Observer {
    void update(int health);
}
```

```
class HealthBar implements Observer {
    public void update(int health) {
        System.out.println(
            "Здоровье на экране: " + health
        );
    }
}
```

```
class Hero {
    private int health = 100;
    private List<Observer> observers = new ArrayList<>();

    void subscribe(Observer observer) {
        observers.add(observer);
    }

    void takeDamage(int damage) {
        health -= damage;

        for (Observer observer : observers) {
            observer.update(health);
        }
```

Где-то в коде:

```
Hero hero = new Hero();
hero.subscribe(new HealthBar());

hero.takeDamage(25);
```

**Преимущество**: интерфейс, журнал событий и другие системы могут реагировать на изменения героя независимо друг от друга.

# Observer
# наблюдатель

IThub

✅ издатели не зависят от конкретных классов подписчиков и наоборот

✅ вы можете подписывать и отписывать получателей на лету

✅ реализует принцип открытости/закрытости

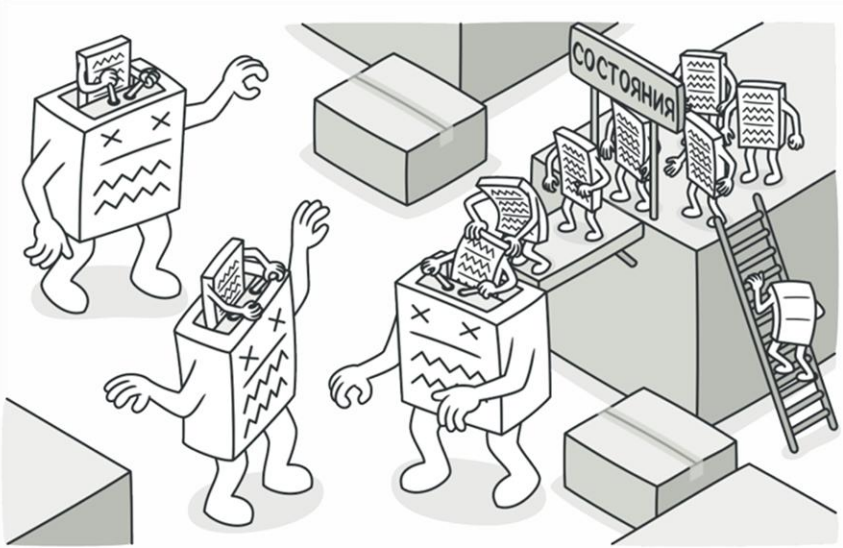
❌ подписчики оповещаются в случайном порядке

# State
# состояние

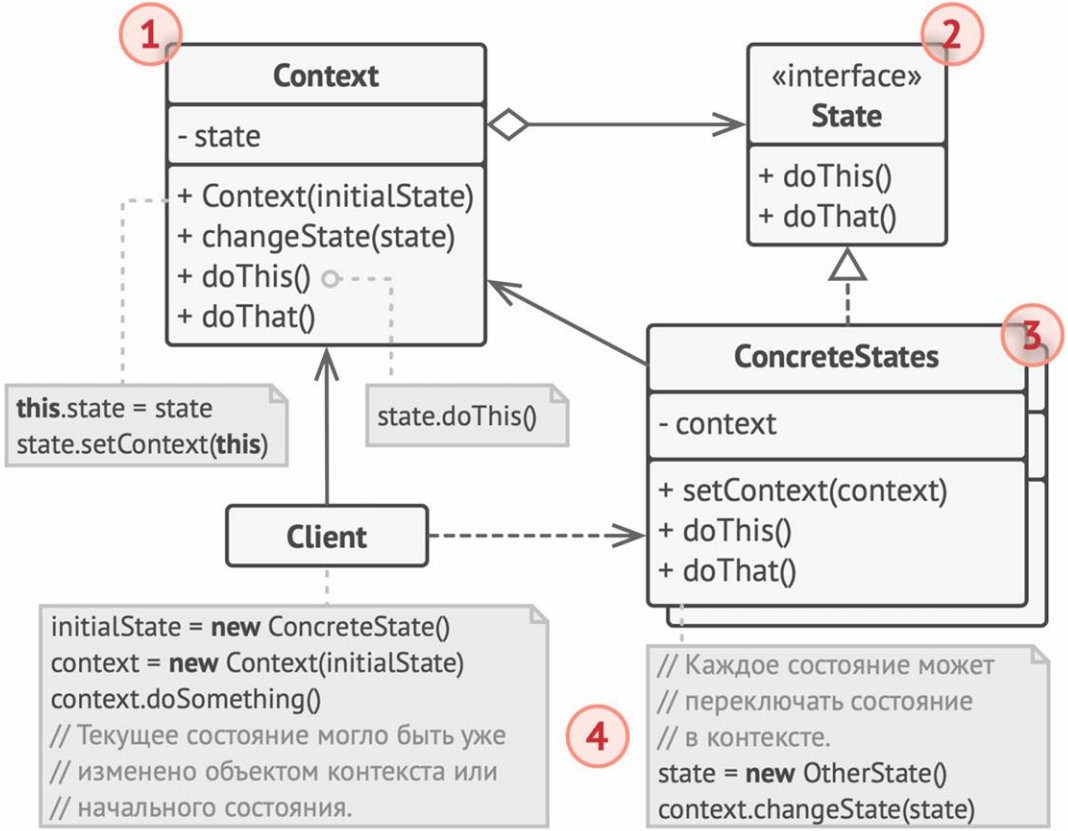
IThub

**Состояние** — это поведенческий паттерн проектирования, который позволяет объектам менять поведение в зависимости от своего состояния.

**Идея**: изменять поведение объекта в зависимости от его текущего состояния.



IThub

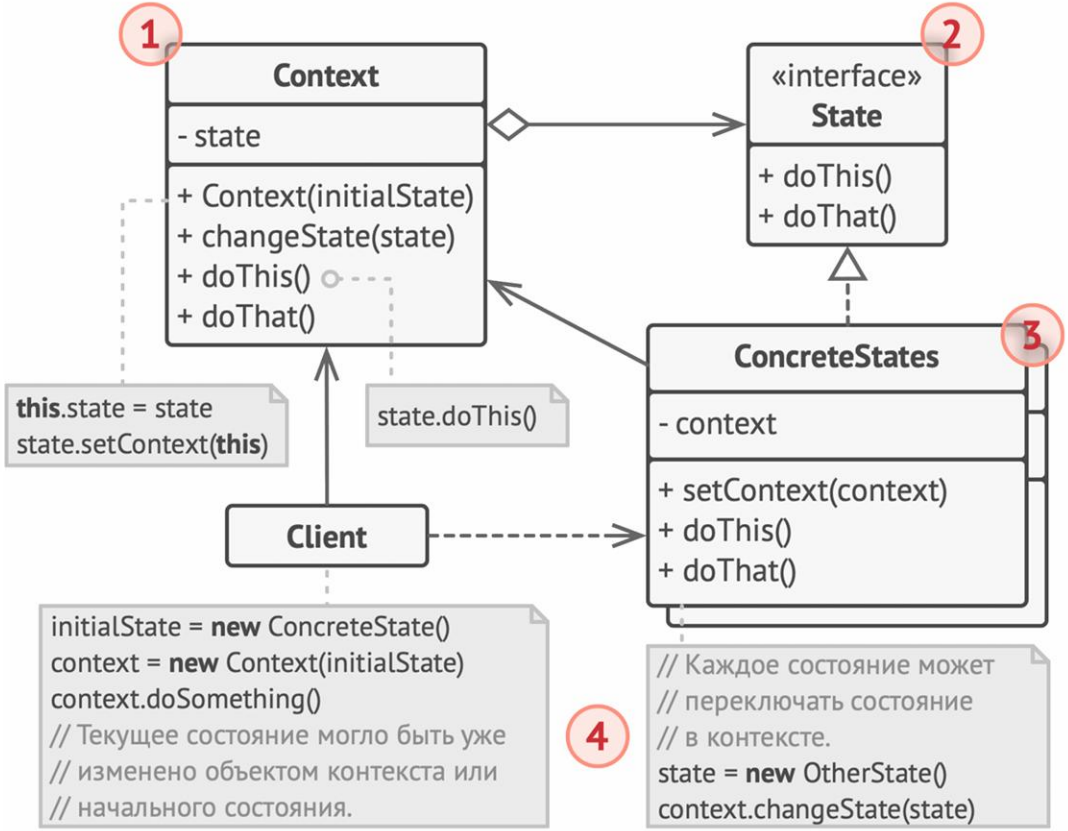



1. **Контекст** хранит ссылку на объект состояния и делегирует ему часть работы, зависящей от состояний.

Контекст работает с этим объектом через общий интерфейс состояний.

Контекст должен иметь метод для присваивания ему нового объекта-состояния.

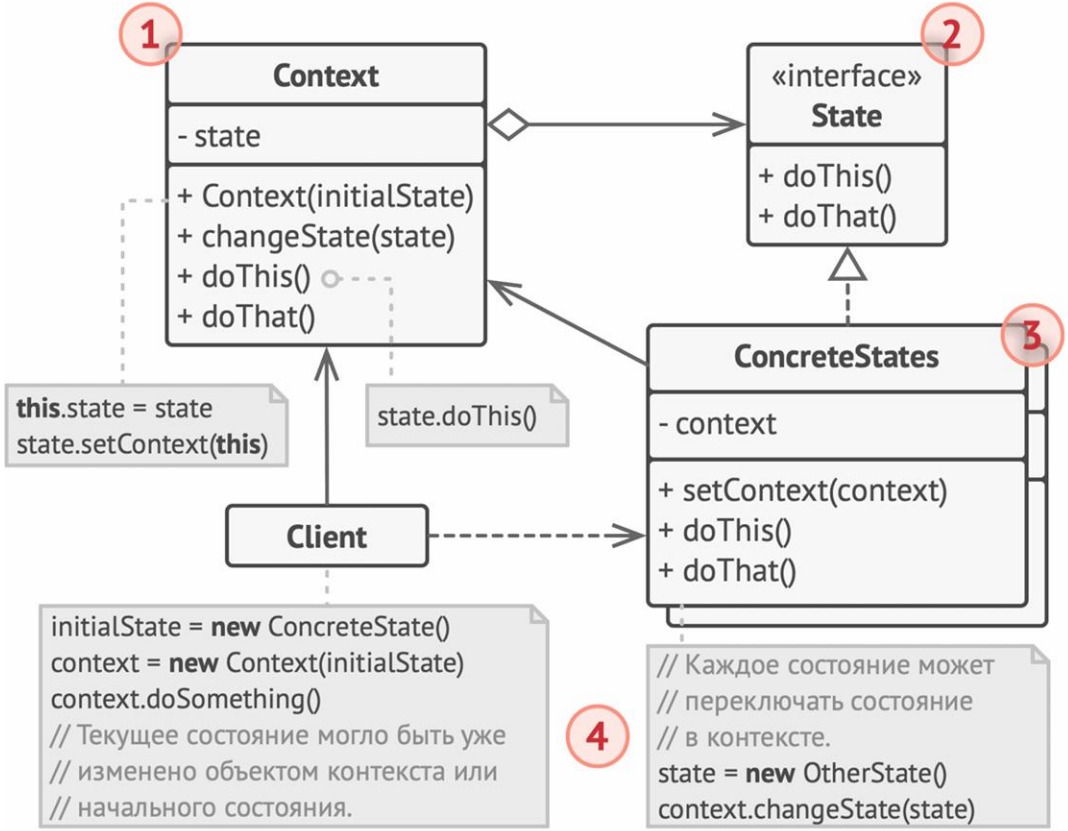
IThub




2. **Состояние** описывает общий интерфейс для всех конкретных состояний.

3. **Конкретные состояния** реализуют поведения, связанные с определённым состоянием контекста.

Иногда приходится создавать целые иерархии классов состояний, чтобы обобщить дублирующий код.




4. И контекст, и объекты конкретных состояний могут решать, когда и какое следующее состояние будет выбрано.

Чтобы переключить состояние, нужно подать другой объект-состояние в контекст.

# Например

IThub

в RPG герой может быть обычным, оглушённым или мёртвым.
Оглушённый персонаж не может атаковать, а мёртвый не может драться.

```
interface HeroState {
    void act();
}
```

```
class NormalState implements HeroState {
    public void act() {
        System.out.println("Герой атакует!");
    }
}
```

```
class StunnedState implements HeroState {
    public void act() {
        System.out.println("Герой оглушён!");
    }
}
```

```
class Hero {
    private HeroState state;

    public Hero () {
         this.state = new NormalState();
    }

    void setState(HeroState state) {
        this.state = state;
    }

    void act() {
        state.act();
    }
}
```

Где-то в коде:

```
Hero hero = new Hero();

hero.act();

hero.setState(new StunnedState());
hero.act();
```

**Преимущество**: логика каждого состояния находится в отдельном классе, поэтому не приходится создавать множество вложенных условий if-else.

# State
# состояние

IThub

✅ избавляет от множества больших условных операторов

✅ концентрирует в одном месте код, связанный с определённым состоянием

❌ может неоправданно усложнить код, если состояний мало и они редко меняются

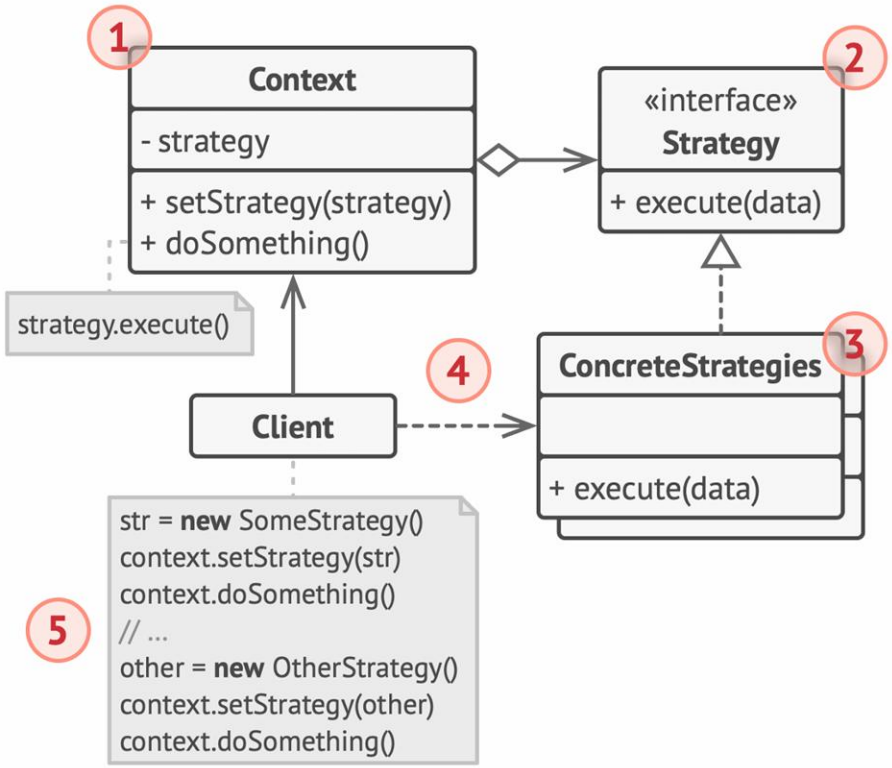
# Strategy
# стратегия

IThub

Стратегия — это поведенческий паттерн проектирования, который определяет семейство схожих алгоритмов и помещает каждый из них в собственный класс, после чего алгоритмы можно взаимозаменять прямо во время исполнения программы.



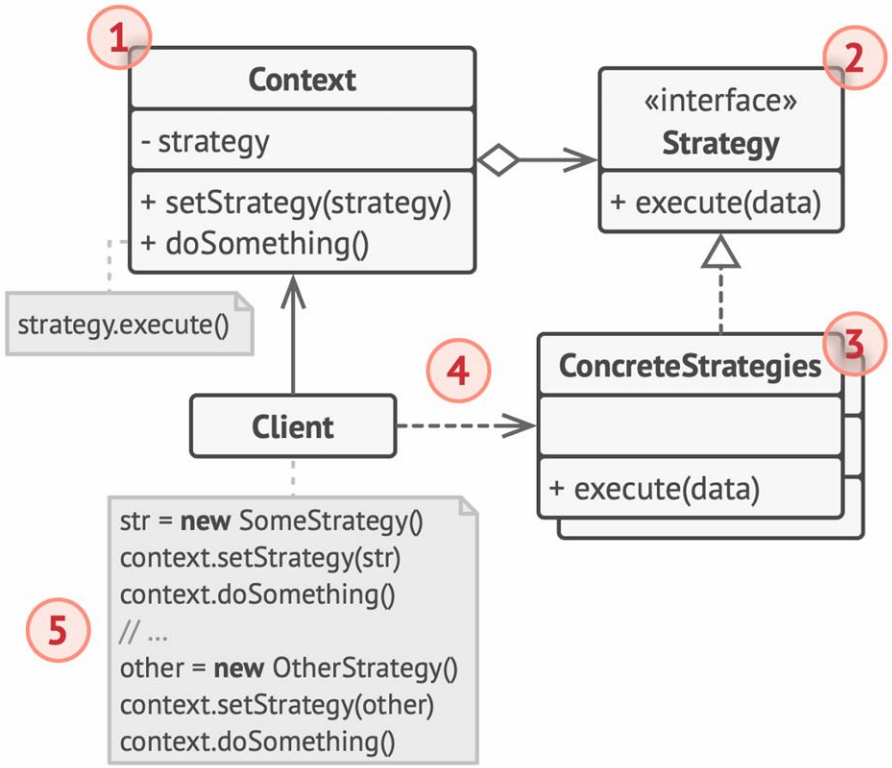
IThub




1. **Контекст** хранит ссылку на объект конкретной стратегии, работая с ним через общий интерфейс стратегий.

2. **Стратегия** определяет интерфейс, общий для всех вариаций алгоритма.

Контекст использует этот интерфейс для вызова алгоритма.




3. **Конкретные стратегии** реализуют различные вариации алгоритма.

4. Во время выполнения программы контекст получает вызовы от клиента и делегирует их объекту конкретной стратегии.

5. Клиент должен создать объект конкретной стратегии и передать его в конструктор контекста.

Кроме этого, клиент должен иметь возможность **заменить стратегию на лету**, используя **сеттер**.

# Например

IThub

в RPG разные персонажи могут использовать разные боевые стили: воин атакует мечом, лучник стреляет из лука, а маг применяет заклинания.

```
interface AttackStrategy {
    void attack();
}
```

```
class SwordAttack implements AttackStrategy {
    public void attack() {
        System.out.println("Удар мечом!");
    }
}
```

```
class BowAttack implements AttackStrategy {
    public void attack() {
        System.out.println("Выстрел из лука!");
    }
}
```

```
class Hero {
    private AttackStrategy strategy;

    Hero(AttackStrategy strategy) {
        this.strategy = strategy;
    }

    void setStrategy(AttackStrategy strategy) {
        this.strategy = strategy;
    }

    void attack() {
        strategy.attack();
    }
}
```

Где-то в коде:

```
Hero hero = new Hero(new SwordAttack());
hero.attack();

hero.setStrategy(new BowAttack());
hero.attack();
```

**Преимущество**: боевую стратегию можно менять во время игры, не переписывая класс героя.

# Strategy
# стратегия

IThub

✅ горячая замена алгоритмов на лету

✅ изолирует код и данные алгоритмов от остальных классов

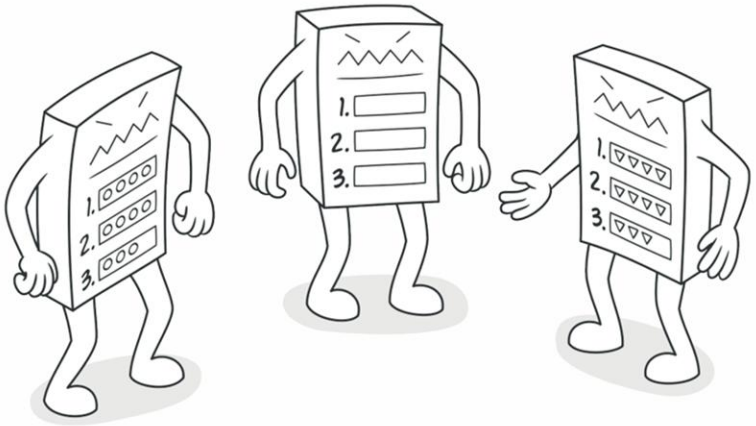
✅ уход от наследования к делегированию

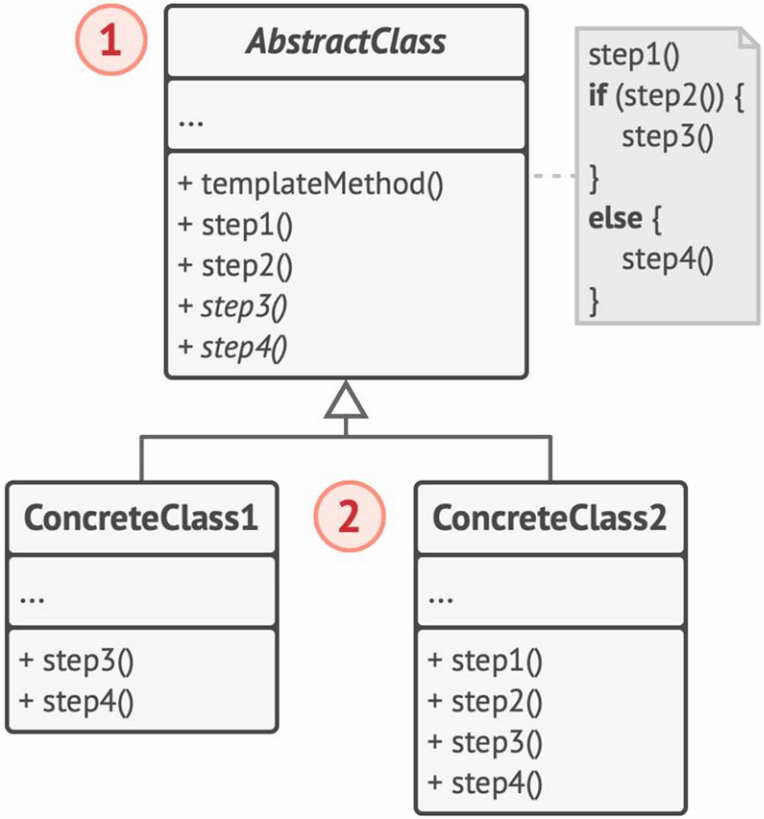
❌ клиент должен знать, в чём состоит разница между стратегиями, чтобы выбрать подходящую

# Template Method
# шаблонный метод

IThub

Шаблонный метод — это поведенческий паттерн проектирования, который определяет скелет алгоритма, перекладывая ответственность за некоторые его шаги на подклассы. Паттерн позволяет подклассам переопределять шаги алгоритма, не меняя его общей структуры.






1. **Абстрактный класс** определяет шаги алгоритма и содержит шаблонный метод, состоящий из вызовов этих шагов.

Шаги могут быть как абстрактными, так и содержать реализацию по умолчанию.

2. **Конкретный класс** переопределяет некоторые (или все) шаги алгоритма.

Конкретные классы не переопределяют сам шаблонный метод.

IThub

# Например

в RPG бой с любым противником состоит из одинаковых этапов: подготовка, атака и завершение.
При этом разные монстры атакуют по-разному.

```
abstract class Monster {
    // шаблонный метод
    void fight() {
        prepare();
        attack();
        finish();
    }

    void prepare() {
        System.out.println("Монстр готовится к бою");
    }

    abstract void attack();

    void finish() {
        System.out.println("Ход монстра завершён");
    }
}
```

IThub

```
class Dragon extends Monster {
    @Override
    void attack() {
        System.out.println("Дракон извергает огонь!");
    }
}
```

```
class Goblin extends Monster {
    @Override
    void attack() {
        System.out.println("Гоблин бьёт кинжалом!");
    }
}
```

Где-то в коде:

```
Monster monster = new Dragon();
monster.fight();

monster = new Goblin();
monster.fight();
```

**Преимущество**: общий порядок действий сохраняется, но отдельные этапы могут отличаться у разных противников.

# Template Method
# шаблонный метод

IThub

✅ облегчает повторное использование кода

❌ вы жёстко ограничены скелетом существующего алгоритма

❌ вы можете нарушить принцип подстановки Барбары Лисков, изменяя базовое поведение одного из шагов алгоритма через подкласс

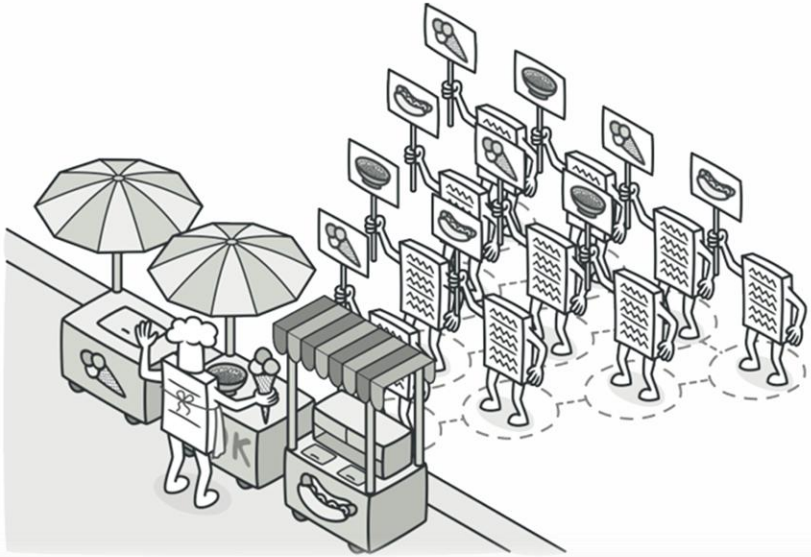
❌ с ростом количества шагов шаблонный метод становится слишком сложно поддерживать

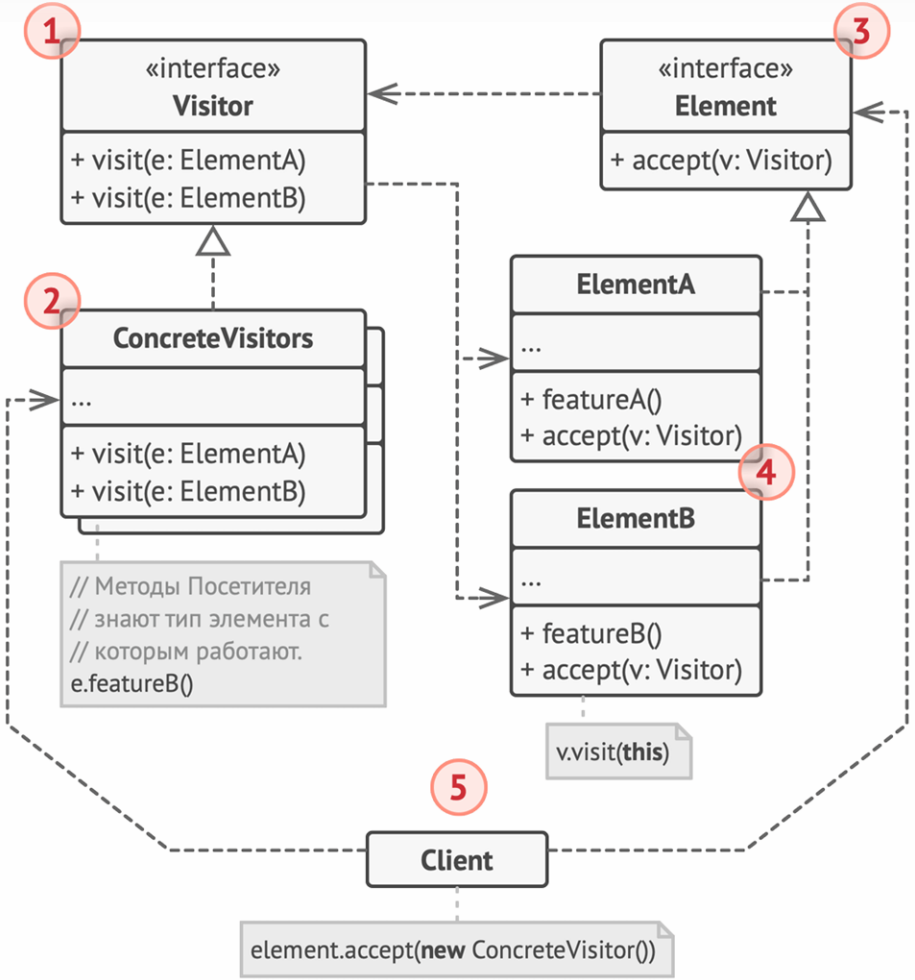
# Visitor
# посетитель

IThub

Посетитель — это поведенческий паттерн проектирования, который позволяет добавлять в программу новые операции, не изменяя классы объектов, над которыми эти операции могут выполняться.

Идея: добавлять новые операции для группы объектов, не изменяя классы этих объектов.



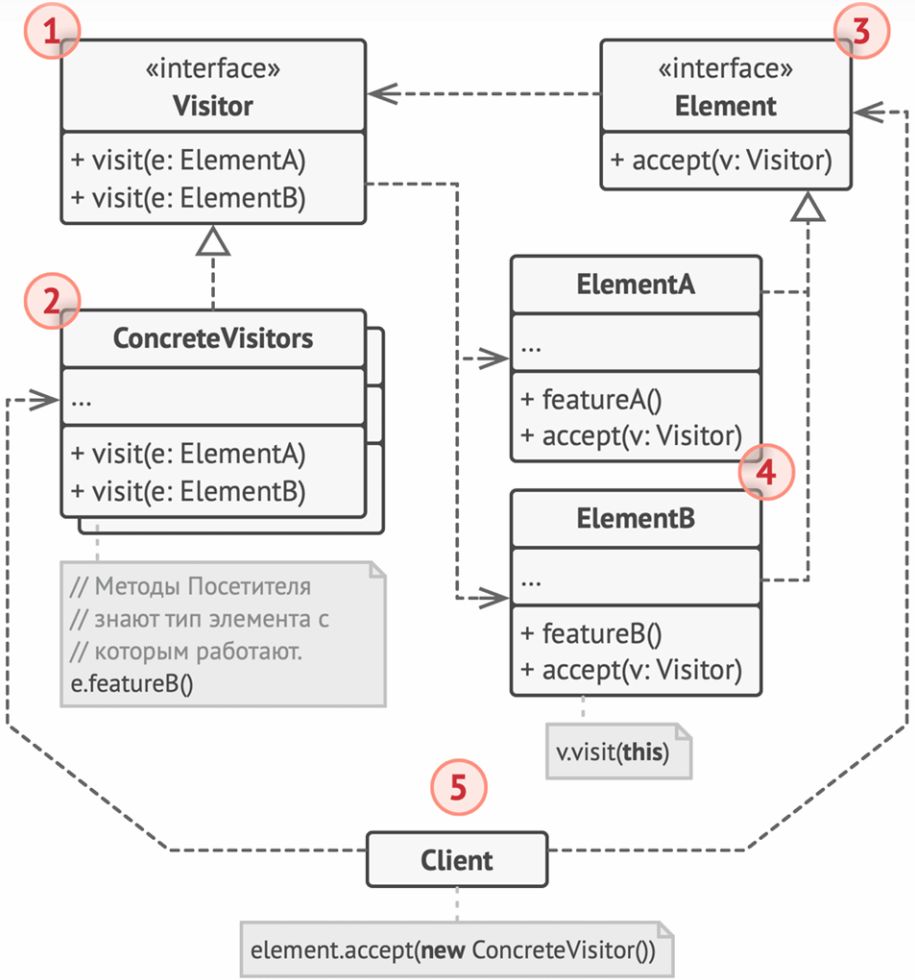



IThub

1. **Посетитель** описывает общий интерфейс для всех типов посетителей.

Он объявляет набор методов, отличающихся типом входящего параметра, которые нужны для запуска операции для всех типов конкретных элементов.

В языках, поддерживающих перегрузку методов, эти методы могут иметь одинаковые имена, но типы их параметров должны отличаться.

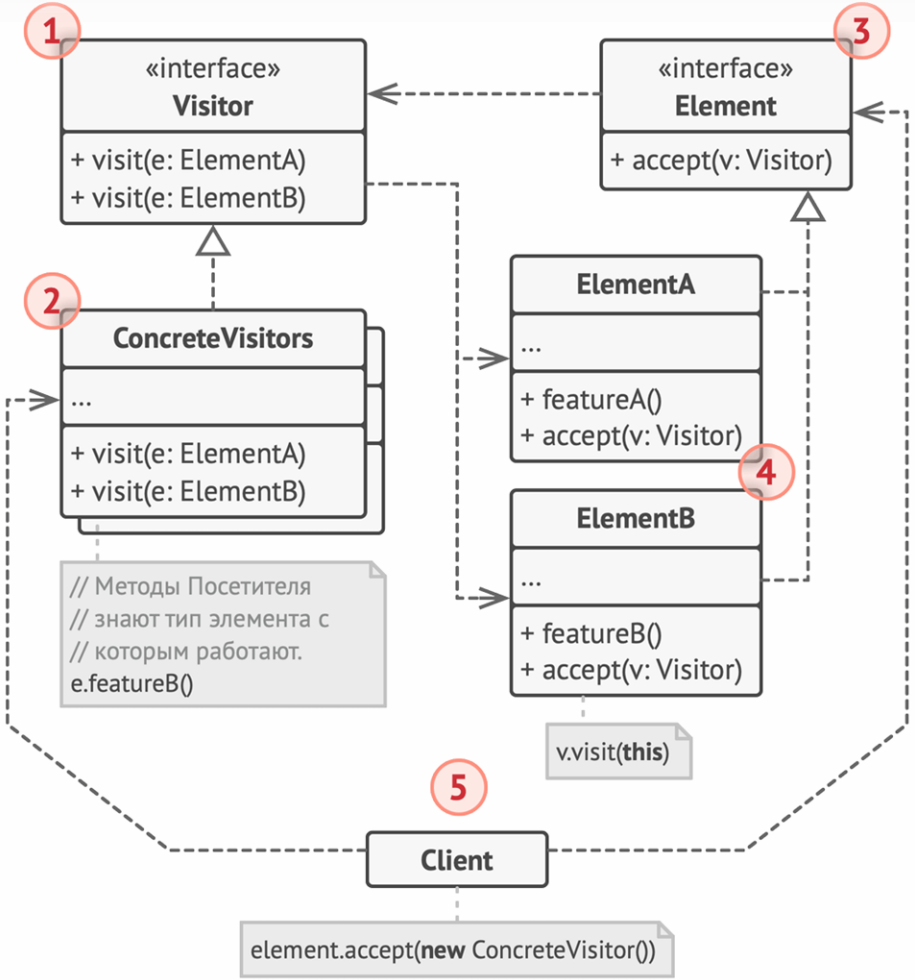



IThub

2. **Конкретные посетители** реализуют какое-то особенное поведение для всех типов элементов, которые можно подать через методы интерфейса посетителя.

3. **Элемент** описывает метод принятия посетителя.

Этот метод должен иметь единственный параметр, объявленный с типом интерфейса посетителя.




IThub

4. **Конкретные элементы** реализуют методы принятия посетителя.

Цель этого метода — вызвать тот метод посещения, который соответствует типу этого элемента.

Так посетитель узнает, с каким именно элементом он работает.

5. **Клиентом** зачастую выступает коллекция или сложный составной объект.

IThub

# Например

в RPG есть разные существа: драконы, гоблины и скелеты.

Например, нужно рассчитать награду за победу над каждым из них.

Вместо добавления метода расчёта в каждый класс можно использовать посетителя.

```
interface Monster {
    void accept(MonsterVisitor visitor);
}
```

```
interface MonsterVisitor {
    void visit(Dragon dragon);
    void visit(Goblin goblin);
}
```

IThub

```
class Dragon implements Monster {
    public void accept(MonsterVisitor visitor) {
        visitor.visit(this);
    }
}
```

```
class Goblin implements Monster {
    public void accept(MonsterVisitor visitor) {
        visitor.visit(this);
    }
}
```

```
class RewardVisitor implements MonsterVisitor {
    public void visit(Dragon dragon) {
        System.out.println("Награда: 100 золота");
    }

    public void visit(Goblin goblin) {
        System.out.println("Награда: 10 золота");
    }
}
```

Где-то в коде:

```
Monster monster = new Dragon();
monster.accept(new RewardVisitor());
```

**Преимущество**: можно добавлять новые операции, например расчёт опыта, оценку силы или формирование статистики, не изменяя классы существ.

Однако при добавлении новых типов монстров интерфейс посетителя придётся расширить.

# Visitor
# посетитель

IThub

✅ упрощает добавление операций, работающих со сложными структурами объектов

✅ Посетитель может накапливать состояние при обходе структуры элементов

❌ паттерн не оправдан, если иерархия элементов часто меняется

# Задание (КТ №8)

IThub

Создать (продолжить) проект, в котором реализован и практически применен **КАЖДЫЙ** из поведенческих паттернов проектирования:

1. Chain of Responsibility
2. Command
3. Interpreter
4. Iterator
5. Mediator
6. Memento
7. Observer
8. State
9. Strategy
10. Template Method
11. Visitor

**! код должен быть покрыт unit-тестами (junit)**

**! проект размещается на GitLab\GitHub с настроенным pipeline на тесты**